

LAPORAN PROYEK ANALISIS SINYAL BIOMEDIS T1
G-Track: Rancang Bangun Sepatu Biomedis untuk Analisis Gait dan
Pemantauan Kesehatan



Kelompok 5

Raihan Ainur Rochim	(185221010)
Nanda Anindya Kusumawati	(185221012)
Amelia Fatikhah	(185221013)
Andika Putra Wijaya Kusuma	(185221014)
Greestaviola Allodya Darmawan	(185221016)
Brigade Mahendra Dharmalaksana	(185221066)

PROGRAM STUDI S1 TEKNIK BIOMEDIS
DEPARTEMEN FISIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS AIRLANGGA
2025

Abstrak

Osteoarthritis (OA) merupakan penyakit sendi degeneratif kronis yang menyebabkan nyeri, kekakuan, dan keterbatasan gerak, terutama pada populasi lansia. Deteksi dini OA menjadi tantangan karena gejala klinis sering muncul saat kerusakan sendi telah lanjut. Proyek ini mengembangkan G-Track, sebuah sepatu pintar berbasis sensor inersia MPU-6050 dan mikrokontroler ESP32, untuk menganalisis pola gait sebagai indikator awal OA. Data percepatan dan kecepatan sudut dikumpulkan dan diproses secara real-time untuk menghitung parameter seperti kecepatan berjalan (*gait velocity*) dan simetri gerak. Algoritma *Decision Tree* diterapkan pada data yang telah diproses untuk mengklasifikasikan kondisi gait pengguna menjadi tiga kelas: Normal & Jalan, Normal & Lari, dan Abnormal. Model menunjukkan akurasi sebesar 99,7% dengan nilai *F1-score*, presisi, dan *recall* masing-masing 1,00. Sistem ini juga dilengkapi antarmuka web yang memungkinkan pemantauan data sensor dan klasifikasi gerakan secara langsung. Hasil menunjukkan bahwa G-Track berpotensi menjadi alat non-invasif untuk skrining dini OA dan pemantauan rehabilitasi secara berkelanjutan.

Kata kunci: Osteoarthritis, gait analysis, MPU-6050, ESP32, Decision Tree, wearable sensor.

I. PENDAHULUAN

1.1 Latar Belakang

Hingga saat ini Osteoarthritis masih menjadi permasalahan penting di seluruh dunia. Osteoarthritis (OA) merupakan penyakit sendi degeneratif kronis yang ditandai dengan kerusakan progresif pada tulang rawan sendi. Kondisi ini sering menyebabkan nyeri, kekakuan, dan keterbatasan gerak, terutama pada sendi penopang berat seperti lutut, panggul, dan tangan (Halodoc, 2024). Menurut Global Burden of Disease, OA telah memengaruhi sekitar 595 juta orang pada tahun 2020, atau sekitar 7,6% dari populasi dunia, dan diproyeksikan terus meningkat. Di Indonesia sendiri, prevalensi OA cukup signifikan, dengan data tahun 2018 menunjukkan 8,5% pada wanita dan 6,1% pada pria, menjadikannya salah satu penyebab disabilitas utama pada lansia (Nisa & Prabandari, 2018; Sari, 2020).

OA berkembang dari ketidakseimbangan antara proses perbaikan dan degradasi tulang rawan sendi. Faktor-faktor seperti usia lanjut, obesitas, riwayat cedera sendi, dan genetik berkontribusi pada patogenesisnya (Sukmana & Suryajaya, 2019; Ningsih, 2021). Kerusakan tulang rawan memicu respons inflamasi, perubahan pada tulang di bawah sendi (subkondral), dan pembentukan osteofit, yang semuanya berkontribusi pada gejala klinis (Sukmana & Suryajaya, 2019). Penyakit ini dapat diklasifikasikan berdasarkan tingkat keparahannya, dari stadium awal tanpa gejala signifikan hingga stadium lanjut dengan kerusakan sendi parah dan kehilangan fungsi (Ningsih, 2021).

Hingga saat ini, deteksi dini osteoarthritis masih menjadi tantangan signifikan, mengingat seringkali gejala awal tidak spesifik atau baru muncul pada tahap lanjut ketika kerusakan sendi sudah terjadi (Hunter & Bierma-Zeinstra, 2019). Metode diagnostik konvensional seperti pencitraan radiografi baru dapat mendeteksi perubahan struktural pada sendi ketika penyakit telah berkembang secara signifikan, yang seringkali terlambat untuk intervensi pencegahan kerusakan lebih lanjut (Kraus et al., 2015; Neogi & Zhang, 2013). Meskipun pendekatan seperti pemeriksaan fisik dan anamnesis dapat memberikan petunjuk, sifat subjektifnya membatasi kemampuan untuk memantau perubahan halus secara objektif seiring waktu.

Keterbatasan dalam deteksi dini ini berdampak pada efektivitas intervensi preventif dan terapeutik, karena penanganan sering kali dimulai ketika kerusakan sudah ireversibel, yang kemudian hanya berfokus pada manajemen gejala seperti nyeri dengan obat-obatan (NSAID atau kortikosteroid) dan terapi fisik (fisioterapi), atau tindakan invasif seperti operasi penggantian sendi (Martel-Pelletier et al., 2016). Oleh karena itu, kebutuhan akan metode deteksi dini yang objektif dan non-invasif menjadi krusial untuk memungkinkan intervensi yang lebih efektif sebelum kerusakan sendi menjadi parah (Glyn-Jones et al., 2015).

Mengingat keterbatasan tersebut, project ini mengusulkan inovasi dalam deteksi dini dan pemantauan progresivitas OA melalui analisis gaya berjalan berbasis kecepatan dan koordinat. Inovasi ini memanfaatkan sensor giroskop MPU 6050 yang terintegrasi dengan ESP32. MPU 6050 adalah Modul Pengukuran Inersia (IMU) yang memiliki fitur akselerometer (untuk mengukur percepatan linear) dan giroskop (untuk mengukur kecepatan sudut), yang mampu menangkap data gerakan enam dimensi (6-DoF) (Chen et al., 2016). Data mentah dari MPU 6050 akan diolah oleh mikrokontroler ESP32—sebuah platform berkemampuan Wi-Fi dan Bluetooth—untuk mengestimasi parameter gaya berjalan seperti kecepatan berjalan, panjang langkah (stride length), lebar langkah (step width), dan simetri langkah (Tekscan, n.d.; Wang & Chen, 2018). Kemampuan ESP32 untuk memproses data secara real-time dan mentransmisikannya secara nirkabel memungkinkan pemantauan yang objektif dan berkelanjutan terhadap perubahan gaya berjalan yang spesifik terkait dengan OA, seperti penurunan kecepatan atau perubahan koordinasi (Hussain et al., 2017; Wang & Chen, 2018).

1.2 Tujuan

1.2.1 Tujuan Umum

Menciptakan sepatu pemantau analisis gait berbasis biomaterial yang inovatif untuk deteksi dini perubahan pola gait, pemantauan rehabilitasi, dan peningkatan performa gerak.

1.2.2 Tujuan Khusus

1. Mengembangkan sistem sensor berbasis biomaterial yang terintegrasi pada sepatu untuk akuisisi data pola gait secara akurat.
2. Merancang algoritma untuk mendeteksi dan menganalisis perubahan pola gait yang menjadi indikator awal dari osteoarthritis.
3. Mengimplementasikan fitur pemantauan real-time untuk mengevaluasi perkembangan pemulihan pasien dan mengoreksi pola berjalan.
4. Menyediakan respon objektif terhadap pola berjalan untuk membantu atlet dalam mengoptimalkan performa dan mencegah cedera.

1.3 Manfaat

1.3.1 Bagi Mahasiswa

1. Memberikan pengalaman praktis dalam perancangan pengembangan dan pengujian sistem wearable device, analisis data sensor, serta integrasi biomaterial dengan teknologi elektronika.
2. Mendorong mahasiswa untuk berpikir kreatif dalam mencari solusi teknologi untuk masalah kesehatan dan performa manusia.

1.3.2 Bagi Masyarakat

1. Menyediakan alat yang non-invasif dan mudah digunakan untuk deteksi dini perubahan pola berjalan yang dapat menjadi indikasi awal berbagai penyakit, memungkinkan intervensi yang lebih cepat.
2. Membantu pasien dalam proses rehabilitasi dengan pemantauan objektif terhadap pola berjalan, mempercepat pemulihan, dan mengurangi risiko komplikasi.
3. Bagi individu yang rentan atau sedang dalam masa pemulihan, alat ini dapat meningkatkan kemandirian dan kepercayaan diri dalam beraktivitas.

1.3.3 Bagi Perguruan Tinggi

1. Menghasilkan inovasi dan publikasi ilmiah yang relevan dengan kebutuhan masyarakat, meningkatkan reputasi perguruan tinggi di bidang penelitian dan pengembangan teknologi kesehatan.

2. Menghasilkan project yang dapat diintegrasikan ke dalam materi perkuliahan, memperkaya studi kasus nyata dan teknologi terkini.

II. TINJAUAN PUSTAKA

2.1 *Decision Tree*

Decision Tree (DT) merupakan salah satu algoritma pembelajaran mesin terawasi (*supervised learning*) yang sangat banyak digunakan karena kemampuannya yang intuitif dalam proses klasifikasi maupun regresi. Algoritma ini membentuk struktur berbentuk pohon, di mana setiap simpul internal merepresentasikan sebuah pengujian terhadap atribut (fitur), setiap cabang mewakili hasil dari pengujian tersebut, dan setiap daun merepresentasikan label kelas akhir. DT sering digunakan dalam sistem pengambilan keputusan karena sifatnya yang mudah dipahami dan divisualisasikan. Pada konteks klasifikasi pola gait berbasis data dari sensor *gyroscope* dan akselerometer, DT memiliki beberapa keunggulan penting. Pertama, algoritma ini dapat menangani relasi non-linier antar fitur dengan baik, dan tidak sensitif terhadap outlier maupun data yang tidak terdistribusi secara normal. Kedua, DT memungkinkan pemilihan fitur-fitur paling berpengaruh dalam proses klasifikasi karena menggunakan teknik pemisahan data berbasis indeks seperti Gini Index atau Information Gain.

Fraiwani dan Hassanin (2021) dalam studi mereka menunjukkan bahwa kombinasi *ensemble* dari *decision tree* seperti AdaBoost, RUSBoost, dan Random Subspace dapat meningkatkan akurasi deteksi penyakit neurodegeneratif (DNDs) berdasarkan analisis sinyal gait *Vertical Ground Reaction Force* (VGRF) hingga mencapai akurasi 99.17% . Studi ini juga menekankan pentingnya pemilihan fitur statistik sederhana seperti *Root Mean Square* (RMS), varians, *skewness*, dan kurtosis, yang terbukti mampu membedakan antara pola gait pasien ALS, Parkinson, Huntington, dan individu sehat dengan sangat baik.

Dalam skenario yang lebih spesifik seperti prediksi independensi gait pasien stroke, algoritma *decision tree* juga telah terbukti berguna dalam mengembangkan *Clinical Prediction Rules* (CPRs). Keunggulan utama DT dalam bidang klinis adalah kemampuannya menyajikan logika pengambilan keputusan dalam bentuk yang mudah ditafsirkan oleh tenaga medis. Penelitian sebelumnya menunjukkan bahwa penggunaan data gait yang dikumpulkan mendekati waktu pulang dari rumah sakit memiliki nilai prediktif yang lebih tinggi dibandingkan data yang diambil saat awal rawat inap, namun

pendekatan menggunakan DT tetap memungkinkan analisis efektif dari data awal tersebut untuk membantu perencanaan rehabilitasi yang lebih personal dan adaptif.

Keandalan model decision tree dapat ditingkatkan dengan pendekatan *ensemble learning*, seperti *Bagging* dan *Boosting*. Dalam pendekatan *Boosting*, seperti AdaBoost, model *decision tree* sederhana dilatih secara berurutan untuk memperbaiki kesalahan dari model sebelumnya, sehingga menghasilkan model akhir yang kuat dan akurat. Ini sangat bermanfaat dalam klasifikasi data gait yang cenderung kompleks dan beragam antar individu. Dengan mempertimbangkan kemampuan interpretasi, efisiensi komputasi, serta kinerja klasifikasi yang kompetitif, decision tree menjadi salah satu metode yang ideal dalam pengembangan sistem cerdas berbasis data sensor wearable untuk klasifikasi pola gait dan deteksi gangguan neurologis.

2.2 Osteoarthritis

Osteoarthritis adalah penyakit sendi degeneratif kronis yang menjadi penyebab utama nyeri sendi, kekakuan, dan disabilitas di seluruh dunia. Kondisi ini dicirikan oleh kerusakan progresif pada tulang rawan sendi, pertumbuhan osteofit (tulang baru di tepi sendi), dan perubahan pada tulang subkondral (tulang di bawah tulang rawan). Osteoarthritis sering menyerang sendi yang menopang berat badan seperti lutut, panggul, tulang belakang, dan sendi-sendi kecil pada tangan dan kaki. Hingga saat ini, Osteoarthritis memengaruhi sekitar 7,6% dari populasi dunia, dan diproyeksikan terus meningkat. Di Indonesia sendiri, prevalensi Osteoarthritis cukup signifikan, menjadikannya salah satu penyebab disabilitas utama pada lansia. (Amalia, dkk., 2023)

Patogenesis Osteoarthritis bersifat multifaktorial, melibatkan interaksi kompleks antara faktor genetik, mekanik, metabolik, dan inflamasi. Seiring bertambahnya usia, kemampuan sel-sel tulang rawan (kondrosit) untuk memperbaiki diri menurun, yang menyebabkan ketidakseimbangan antara proses sintesis dan degradasi matriks ekstraseluler tulang rawan. Beban mekanis yang berlebihan atau berulang, seperti pada obesitas atau riwayat cedera sendi sebelumnya, mempercepat kerusakan tulang rawan. Selain itu, faktor inflamasi tingkat rendah dan gangguan metabolisme juga berperan dalam perkembangan penyakit. Kerusakan tulang rawan memicu respons inflamasi,

perubahan pada tulang di bawah sendi, dan pembentukan osteofit, yang semuanya berkontribusi pada gejala klinis. Penyakit ini dapat diklasifikasikan berdasarkan tingkat keparahannya, dari stadium awal tanpa gejala signifikan hingga stadium lanjut dengan kerusakan sendi parah dan kehilangan fungsi.

Secara klinis, Osteoarthritis bermanifestasi sebagai nyeri sendi yang memburuk dengan aktivitas dan berkurang saat istirahat, kekakuan sendi terutama di pagi hari atau setelah tidak bergerak, keterbatasan gerak, krepitus (bunyi gesekan), dan kadang-kadang pembengkakan sendi. Diagnosis biasanya didasarkan pada kombinasi gejala klinis, pemeriksaan fisik, dan temuan radiografi seperti penyempitan celah sendi, pembentukan osteofit, dan sklerosis subkondral. Namun, perlu dicatat bahwa korelasi antara temuan radiografi dan gejala klinis tidak selalu linier, artinya seseorang dengan kerusakan radiografi yang signifikan mungkin tidak merasakan nyeri yang parah, dan sebaliknya.

Perubahan pada pola gaya berjalan (gait) merupakan indikator penting dan objektif dari progresivitas Osteoarthritis. Ketika sendi mengalami kerusakan dan nyeri, individu cenderung mengubah cara berjalannya untuk mengurangi beban atau rasa sakit, yang dapat termanifestasi sebagai asimetri, penurunan kecepatan, atau perubahan koordinasi. Perubahan ini, meskipun seringkali halus pada tahap awal, dapat dideteksi melalui analisis gait dan memberikan petunjuk berharga mengenai kondisi sendi (Hami, dkk., 2023)

Pada pasien dengan Osteoarthritis lutut, terdapat perbedaan yang signifikan dalam beberapa parameter gait dibandingkan dengan individu sehat, seperti ditunjukkan pada tabel berikut:

Tabel 1. Parameter Gait pembandingan individu sehat dengan pasien Osteoarthritis lutut

Parameter	Pasien Osteoarthritis Lutut	Kelompok Kontrol (Sehat)
Kecepatan gait (m/s)	0.74±0.08	1.27±0.14
<i>Cadence</i> (langkah/menit)	97.8±11.5	116.3±7.4
Simetri tubuh (<i>ML trunk symmetry</i>)	9.4°±2.2°	4.3°±1.9°
Simetri tungkai (<i>angular degree</i>)	11.7°±1.6°	3.1°±2.3°
Simetri tungkai (<i>time, msec</i>)	5.7±2.8	1.2±0.9

Perubahan pada parameter-parameter gait ini mencerminkan upaya tubuh untuk mengurangi nyeri dan kompensasi atas kerusakan sendi, yang pada akhirnya dapat memperburuk kondisi jika tidak ditangani. Penurunan kecepatan berjalan dan *cadence*, serta peningkatan asimetri pada tubuh dan tungkai, sering diamati pada individu dengan Osteoarthritis karena mereka cenderung mengambil langkah yang lebih pendek, lebih lambat, dan tidak seimbang untuk menghindari pembebanan pada sendi yang sakit (Staab, dkk., 2014)

Manajemen Osteoarthritis saat ini berfokus pada pengurangan nyeri, peningkatan fungsi, dan perlambatan progresivitas penyakit. Pendekatan meliputi terapi non-farmakologis (edukasi pasien, penurunan berat badan, latihan fisik), farmakologis (analgesik, NSAID, suntikan intra-artikular), dan pada kasus lanjut,

intervensi bedah seperti artroplasti (penggantian sendi). Tantangan utama dalam penanganan Osteoarthritis adalah deteksi dini, karena kerusakan tulang rawan seringkali sudah lanjut saat gejala muncul, membatasi efektivitas intervensi pencegahan. Oleh karena itu, kebutuhan akan metode deteksi dini yang objektif dan non-invasif, terutama melalui analisis perubahan gaya berjalan, menjadi krusial untuk memungkinkan intervensi yang lebih efektif sebelum kerusakan sendi menjadi parah. (Lih H, 2022)

2.3 *Personalized Medicine*

Personalized medicine atau pengobatan yang dipersonalisasi merupakan pendekatan yang mengadaptasi strategi medis berdasarkan karakteristik unik tiap individu, seperti faktor genetik, gaya hidup, lingkungan, serta respons fisiologis. Dalam bidang deteksi pola gait (*gait analysis*), pendekatan ini menjadi sangat relevan karena variasi pola jalan seseorang dapat mencerminkan kondisi neurologis maupun muskuloskeletal secara spesifik dan personal. Dengan kemajuan teknologi *wearable* dan *machine learning*, analisis gait kini dapat dilakukan melalui sistem yang terintegrasi dengan sensor seperti *gyroscope* dan *accelerometer*. *Wearable devices* ini memungkinkan pengumpulan data biomekanik secara *real-time*, yang kemudian dianalisis menggunakan algoritma kecerdasan buatan untuk mengidentifikasi kelainan pola jalan, mendeteksi dini gangguan motorik, atau mengevaluasi hasil terapi rehabilitasi.

Menurut Olyanasab dan Annabestani (2024), *personalized wearable biomedical devices* telah menjadi landasan baru dalam sistem monitoring kesehatan individu, termasuk dalam klasifikasi data *gyroscope* dan ECG untuk menilai efektivitas terapi pada pasien gangguan pergelangan kaki. Salah satu studi dalam ulasan mereka menggunakan model *Support Vector Machine (SVM)* untuk mengklasifikasikan sinyal *gyro*, membedakan fase awal dan akhir terapi, dengan akurasi mencapai 91.7%. Hal ini menunjukkan bahwa teknologi *wearable* yang dipersonalisasi bukan hanya mampu mendeteksi masalah gait, tapi juga berfungsi sebagai alat evaluasi klinis. Selain aspek deteksi, *personalized medicine* juga memberikan keunggulan dalam hal pemberian

intervensi yang lebih tepat sasaran. Deteksi berbasis sensor memungkinkan sistem untuk mengenali pola-pola individual dalam perubahan gait yang mungkin terjadi akibat penyakit seperti Parkinson, stroke, atau cedera saraf. Sistem tersebut dapat menyarankan jenis terapi, intensitas latihan, atau bahkan modifikasi gaya hidup secara otomatis berdasarkan respons tubuh pengguna.

Sugandh et al. (2023) menekankan bahwa prinsip *personalized medicine* juga dapat diterapkan pada bidang neurologi dan rehabilitasi, termasuk deteksi dan intervensi dini gangguan gerak seperti gait *abnormality*. Teknologi seperti *continuous monitoring* dan *predictive models* memungkinkan pelacakan perubahan secara longitudinal, memberikan panduan adaptif untuk perawatan berkelanjutan.

2.4 Gait

Gait atau pola berjalan manusia merupakan suatu proses pergerakan berulang yang kompleks dan terkoordinasi, melibatkan interaksi antara sistem saraf pusat, sistem muskuloskeletal, serta umpan balik sensorik. Gait berperan penting dalam mobilitas dan keseimbangan tubuh manusia, dan menjadi salah satu indikator utama dalam mengevaluasi fungsi motorik, baik dalam kondisi normal maupun patologis. Dalam praktik klinis, analisis gait digunakan untuk mendeteksi berbagai gangguan neurologis, muskuloskeletal, serta mengevaluasi efektivitas program rehabilitasi.

Pola gait yang normal ditandai dengan gerakan yang simetris antara kaki kiri dan kanan, distribusi beban tubuh yang merata, kecepatan berjalan yang stabil, serta konsumsi energi yang efisien. Menurut kajian oleh Siti Rodiah et al. (2024), siklus gait terdiri dari dua fase utama, yaitu fase *stance* dan fase *swing*. Fase *stance* terjadi ketika kaki menyentuh tanah dan menopang berat badan, sedangkan fase *swing* terjadi saat kaki diangkat dari tanah dan bergerak ke depan untuk langkah berikutnya. Masing-masing fase ini memiliki sub-fase yang mencerminkan dinamika perpindahan tubuh dalam satu siklus berjalan, seperti *initial contact*, *midstance*, hingga *terminal swing*. Parameter penting yang menggambarkan gait normal meliputi panjang langkah (*stride*), jumlah langkah per menit (*cadence*), kecepatan berjalan, keseimbangan tubuh, dan *ground reaction force* (GRF) yang merata selama fase *stance*.

Sebaliknya, gait abnormal terjadi ketika terdapat gangguan pada salah satu atau lebih sistem pengendali gerakan. Gangguan ini dapat disebabkan oleh kondisi seperti stroke, osteoarthritis, cedera ligamen, kelemahan otot, atau penyakit neurodegeneratif seperti Parkinson. Pola gait abnormal biasanya ditandai dengan asimetri gerakan antar kaki, kecepatan berjalan yang menurun, waktu *stance* yang lebih lama pada satu sisi tubuh, serta distribusi gaya tekan kaki terhadap tanah yang tidak merata. Dalam penelitian oleh Donnenfield et al. (2022), ditemukan bahwa peningkatan tekanan puncak (peak pressure) pada kaki yang mengalami operasi ACL berkorelasi dengan kerusakan kartilago yang lebih parah. Hal ini menunjukkan bahwa perubahan distribusi beban saat berjalan dapat memengaruhi outcome klinis secara signifikan.

2.5 Gyroscope

Gyroscope merupakan sensor yang digunakan untuk mengukur kecepatan sudut (*angular velocity*) terhadap tiga sumbu utama (x, y, dan z), dan menjadi komponen vital dalam sistem pemantauan gerakan berbasis *Inertial Measurement Unit* (IMU). Dalam konteks penelitian ini, gyroscope terintegrasi dalam modul MPU-6050, yang merupakan sensor 6-axis berbasis MEMS (*Micro-Electro-Mechanical Systems*) yang menggabungkan akselerometer dan *gyroscope* dalam satu chip. Sensor ini bekerja dengan prinsip dasar *vibrating structure gyroscope*, di mana gerakan rotasi menyebabkan perubahan resonansi mikrostruktur internal yang kemudian dikonversi menjadi sinyal listrik. Modul ini mampu mengukur kecepatan sudut dalam rentang ± 250 , ± 500 , ± 1000 , hingga ± 2000 derajat per detik dengan resolusi 16-bit, serta mendukung *sampling rate* hingga 1 kHz. MPU-6050 dihubungkan ke mikrokontroler Arduino Nano menggunakan antarmuka komunikasi I2C, yang memungkinkan transfer data dua arah secara efisien hanya melalui dua pin (SCL dan SDA). Sinyal yang diperoleh dari *gyroscope* dikirimkan secara *real-time* ke perangkat eksternal menggunakan modul Bluetooth HC-05, memungkinkan sistem untuk merekam dan memantau gerakan tubuh secara nirkabel. (Homes et al., 2023)

Agar hasil pembacaan akurat dan stabil, dilakukan proses kalibrasi awal pada *gyroscope*. Kalibrasi ini dilakukan saat sistem dalam keadaan diam, sehingga nilai

offset (bias) dan *drift* dapat dikoreksi secara manual. Selain itu, pengaturan sensitivitas dan filter digital internal (*Digital Low Pass Filter* – DLPF) pada modul juga disesuaikan untuk mengurangi noise. Implementasi *gyroscope* dalam sistem ini menegaskan pentingnya aspek kestabilan dan keandalan perangkat keras dalam proses akuisisi data untuk aplikasi biomedical, seperti pemantauan postur atau analisis gerakan. Tanpa keakuratan data dari *gyroscope*, sistem berbasis IMU tidak dapat memberikan interpretasi spasial gerakan tubuh secara tepat.

2.6 Accelerometer

Akselerometer adalah sensor yang berfungsi untuk mengukur percepatan linear atau gaya non-gravitasi pada suatu objek dalam tiga dimensi (sumbu x, y, dan z). Sensor ini merupakan komponen krusial dalam sistem pengukuran inersia (IMU) karena kemampuannya dalam mendeteksi perubahan kecepatan dan orientasi gerakan. Prinsip kerja akselerometer modern umumnya didasarkan pada *Micro-Electro-Mechanical Systems* (MEMS), di mana massa mikro disematkan pada pegas yang bergerak saat terjadi percepatan. Perubahan posisi massa ini akan diukur melalui perubahan kapasitansi atau resistansi, yang kemudian dikonversi menjadi sinyal listrik yang merepresentasikan nilai percepatan. (Aviles, dkk., 2019)

Dalam konteks analisis gaya berjalan atau pemantauan aktivitas fisik, akselerometer berperan penting dalam mengidentifikasi parameter gerak seperti kecepatan, jarak, dan pola langkah. Data percepatan yang dikumpulkan dapat diintegrasikan (*double integration*) untuk mengestimasi posisi dan kecepatan, meskipun ini memerlukan filter untuk mengatasi *drift* atau akumulasi kesalahan. Pada aplikasi *wearable device*, akselerometer sering digunakan untuk mendeteksi permulaan dan akhir siklus langkah, serta untuk membedakan antara aktivitas seperti berjalan, berlari, atau berdiri. Keunggulan akselerometer terletak pada ukurannya yang kecil, konsumsi daya yang rendah, dan kemampuan untuk memberikan data gerak secara real-time. (Anshari, dkk., 2025)

Akurasi data akselerometer dapat dipengaruhi oleh *noise* dan *bias*, sehingga kalibrasi dan *filtering* seringkali diperlukan. Kalibrasi dilakukan untuk menghilangkan

offset dan kesalahan skala yang inheren pada sensor, sedangkan filter digital seperti *low-pass filter* atau *Kalman filter* digunakan untuk mengurangi *noise* dan mendapatkan sinyal yang lebih bersih. Penggabungan data dari akselerometer dengan giroskop (melalui *sensor fusion*) memungkinkan estimasi orientasi yang lebih stabil dan akurat, yang esensial untuk analisis gerak tubuh yang kompleks. Dengan demikian, akselerometer menjadi fondasi utama dalam pengembangan sistem cerdas untuk deteksi dini masalah kesehatan dan pemantauan performa gerak manusia.

2.7 Sensor MPU 6050

Sensor MPU-6050 adalah sebuah Modul Pengukuran Inersia (IMU) 6-axis yang sangat populer dan banyak digunakan dalam aplikasi yang membutuhkan pengukuran gerak dan orientasi (Al Fath, dkk.,2018). Modul ini terintegrasi dalam satu *chip* tunggal dan menggabungkan dua sensor penting yaitu akselerometer 3-axis dan giroskop 3-axis. Dengan demikian, MPU-6050 mampu menyediakan data gerakan enam dimensi, yang mencakup percepatan linear di sumbu X, Y, Z, serta kecepatan sudut di sekitar ketiga sumbu tersebut. Spesifikasi utamanya adalah sebagai berikut:

1. Akselerometer: Mampu mengukur percepatan dalam rentang terprogram $\pm 2g, \pm 4g, \pm 8g$, atau $\pm 16g$. Ini memungkinkan deteksi perubahan kecepatan dan orientasi dalam berbagai skala gerak.
2. Giroskop: Mampu mengukur kecepatan sudut dalam rentang terprogram $\pm 250, \pm 500, \pm 1000$, atau ± 2000 derajat per detik (dps). Ini penting untuk mendeteksi rotasi dan perubahan arah.
3. *Digital Motion Processor* (DMP): Salah satu fitur unggulan MPU-6050 adalah adanya DMP internal. DMP adalah prosesor *on-chip* yang dapat melakukan algoritma fusi sensor kompleks untuk menggabungkan data akselerometer dan giroskop. Ini menghasilkan data orientasi yang lebih stabil dan akurat (seperti *roll*, *pitch*, *yaw*) serta mengurangi beban komputasi pada mikrokontroler eksternal.
4. Antarmuka Komunikasi I2C: MPU-6050 berkomunikasi dengan mikrokontroler melalui antarmuka *Inter-Integrated Circuit* (I2C), yang

efisien dan hanya membutuhkan dua pin data (SDA dan SCL) untuk transfer data dua arah.

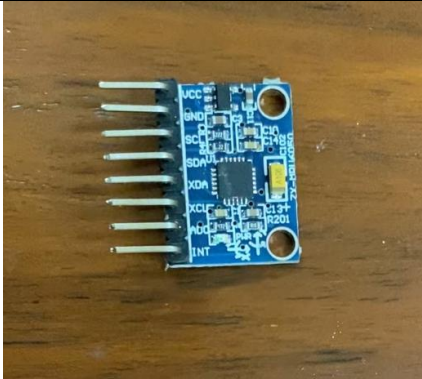
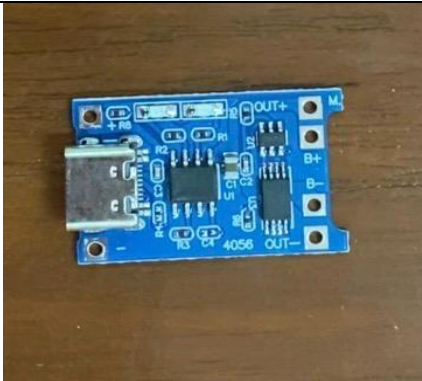
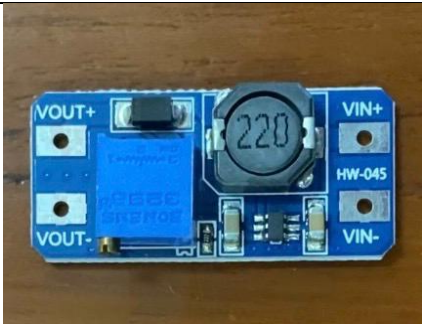
5. Suhu Terintegrasi: Sensor ini juga dilengkapi dengan sensor suhu internal yang dapat digunakan untuk kompensasi suhu pada pembacaan akselerometer dan giroskop, meningkatkan akurasi data.

Dalam konteks proyek analisis gaya berjalan, MPU-6050 menjadi pilihan sensor yang ideal karena kemampuannya menyediakan data gerak yang komprehensif. Data percepatan dari akselerometer dapat digunakan untuk mendeteksi fase-fase langkah (seperti *initial contact* dan *terminal contact*) dan menghitung parameter seperti panjang langkah dan *cadence*. Sementara itu, data kecepatan sudut dari giroskop sangat berguna untuk menganalisis rotasi sendi, stabilitas postur, dan simetri gerak selama berjalan.

Integrasi MPU-6050 dengan mikrokontroler seperti ESP32 memungkinkan akuisisi data *real-time* yang presisi. Data mentah yang diperoleh kemudian dapat diolah menggunakan algoritma khusus untuk mengekstraksi fitur-fitur gait yang relevan dan mendeteksi anomali yang mungkin mengindikasikan kondisi kesehatan tertentu. Keandalan dan akurasi MPU-6050 menjadikannya solusi yang efektif untuk sistem pemantauan gerakan berbasis *wearable device*.

III. METODE

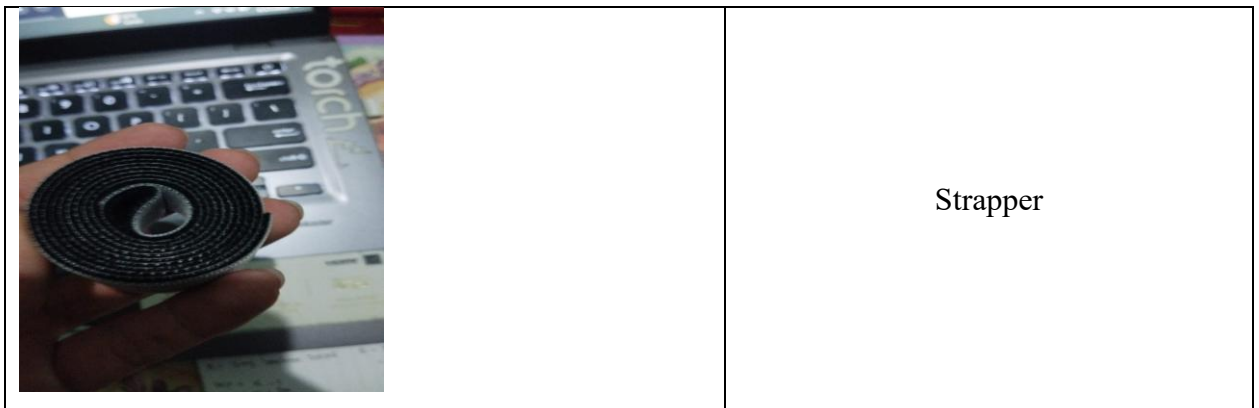
3.1 Alat & Bahan

Foto Komponen	Nama Komponen
	Sensor MPU6050
	Modul Charger
	Modul Step Up DC Voltage

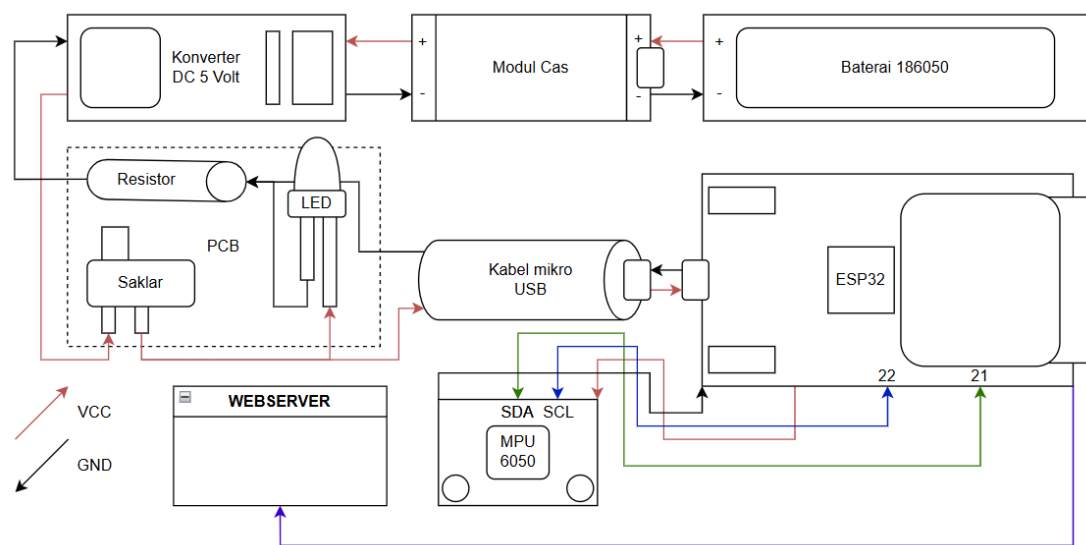
		ESP32
		Baterai 186050
		Kabel Data Micro B
		LED

		PCB
		Sepatu
		Solder
		Resistor 220Ω
		Timah

		Saklar
		Kotak Baterai
		Kotak kardus



3.2 Sistemasi Rangkaian



Sistem ini dirancang sebagai perangkat pendeteksi gerak berbasis ESP32 yang portabel dan dapat diakses melalui web. Sumber daya utama berasal dari Konverter DC 5 Volt yang memberi daya ke Modul Cas. Modul Cas ini bertugas mengisi ulang Baterai 18650, sekaligus menyediakan daya untuk seluruh rangkaian. Daya kemudian dialirkan ke modul ESP32 melalui Kabel mikro USB, yang juga memungkinkan pemrograman dan komunikasi serial. ESP32 sendiri berfungsi sebagai otak utama sistem, yang akan memproses data dan menjadi inti dari Webserver.

Pada intinya, sistem ini memanfaatkan MPU6050 sebagai sensor gerak utama (akselerometer dan giroskop), yang terhubung ke ESP32 melalui antarmuka I2C (SDA ke GPIO 21, SCL ke GPIO 22). Data dari MPU6050 diproses oleh program di ESP32 untuk mendeteksi dan mengklasifikasikan gerakan. Hasil deteksi dan data sensor

lainnya kemudian ditampilkan dan dapat diakses secara nirkabel melalui Webserver yang berjalan di ESP32. Selain itu, terdapat PCB terpisah yang dilengkapi Resistor, LED sebagai indikator visual, dan Saklar untuk kontrol atau input manual, melengkapi fungsi interaksi fisik perangkat ini.

3.3 Kodingan Deceision Tree

Kode full: [[Model Decision Tree](#)]

1. Data Gathering

```
[ ] from google.colab import drive
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

[ ] data=pd.read_csv('/content/sample_data/dataset.csv')
data.head(10)
```

	date	time	username	wrist	activity	acceleration_x	acceleration_y	acceleration_z	gyro_x	gyro_y	gyro_z
0	2017-6-30	13:51:15:847724020	viktor	0	0	0.2650	-0.7814	-0.0076	-0.0590	0.0325	-2.9296
1	2017-6-30	13:51:16:246945023	viktor	0	0	0.6722	-1.1233	-0.2344	-0.1757	0.0208	0.1269
2	2017-6-30	13:51:16:446233987	viktor	0	0	0.4399	-1.4817	0.0722	-0.9105	0.1063	-2.4367
3	2017-6-30	13:51:16:646117985	viktor	0	0	0.3031	-0.8125	0.0888	0.1199	-0.4099	-2.9336
4	2017-6-30	13:51:16:846738994	viktor	0	0	0.4814	-0.9312	0.0359	0.0527	0.4379	2.4922
5	2017-6-30	13:51:17:46806991	viktor	0	0	0.4044	-0.8056	-0.0956	0.6925	-0.2179	2.5750
6	2017-6-30	13:51:17:246767997	viktor	0	0	0.6320	-1.1290	-0.2982	0.0548	-0.1896	0.4473
7	2017-6-30	13:51:17:446566025	viktor	0	0	0.6670	-1.3503	-0.0880	-0.8094	-0.7938	-1.4348
8	2017-6-30	13:51:17:646152973	viktor	0	0	0.2704	-0.8633	0.1293	-0.4173	-0.1904	-2.6759
9	2017-6-30	13:51:17:846502006	viktor	0	0	0.4690	-1.0740	0.0219	0.0388	1.1491	1.6982

Tahap pengumpulan data dalam kode ini dilakukan dengan membaca dataset yang tersimpan secara lokal di *Google Colab* melalui `pd.read_csv('/content/sample_data/dataset.csv')`. Sebelum itu, beberapa *library* penting seperti `numpy`, `pandas`, `matplotlib.pyplot`, dan `seaborn` diimpor untuk keperluan analisis numerik, manipulasi data, serta visualisasi. Data yang ditampilkan hanya berupa 10 data pertama dari dataset, menggunakan `data.head(10)`.

2. Data Preprocessing

a. Mengubah bentuk data

```
[ ] print("Tipe data sebelum konversi:")
print(data['time'].dtype)
print("\nData sebelum konversi:")
print(data[['time']].head())

def time_to_seconds(time_str):
    hh, mm, ss, nano = map(int, time_str.split(':'))
    return hh * 3600 + mm * 60 + ss + nano / 1e9

data['time_float'] = data['time'].apply(time_to_seconds).astype(float)

print("\nData setelah konversi:")
print(data[['time', 'time_float']].head())
print("\nTipe data setelah konversi:")
print(data['time_float'].dtype)

data.drop(columns=['time'], inplace=True)

data.rename(columns={'time_float': 'time'}, inplace=True)
```

Tipe data sebelum konversi:
object

Data sebelum konversi:

	time
0	13:51:15:847724020
1	13:51:16:246945023
2	13:51:16:446233987
3	13:51:16:646117985
4	13:51:16:846738994

Data setelah konversi:

	time	time_float
0	13:51:15:847724020	49875.847724
1	13:51:16:246945023	49876.246945
2	13:51:16:446233987	49876.446234
3	13:51:16:646117985	49876.646118
4	13:51:16:846738994	49876.846739

Tipe data setelah konversi:
float64

Pada kode waktu di atas dilakukan pengubahan bentuk data yang awalnya berupa *string* atau *object* (dalam *format* jam:menit:detik:nanodetik) menjadi nilai float dalam satuan detik. Fungsi `time_to_seconds()` dibuat untuk memisahkan dan mengubah setiap bagian waktu menjadi detik, kemudian hasilnya disimpan dalam kolom baru `time_float`, yang akhirnya menggantikan kolom asli `time` guna memudahkan proses pengolahan fitur.

b. Konversi *accelerometer* menjadi *velocity*

```
[ ] delta_t = data['time'].diff().dropna()

fs = 1 / delta_t.mean()

print(f"Frekuensi sampling: {fs:.2f} Hz")

Frekuensi sampling: 3.91 Hz

[ ] dt = 1 / fs

vx = np.cumsum(data['acceleration_x'] * dt)
vy = np.cumsum(data['acceleration_y'] * dt)
vz = np.cumsum(data['acceleration_z'] * dt)

data['velocity'] = np.sqrt(vx**2 + vy**2 + vz**2)

print(data[['velocity']].head())

velocity
0    0.211256
1    0.547002
2    0.936950
3    1.158071
4    1.425300
```


Pada bagian pertama, dilakukan perhitungan frekuensi sampling (berapa kali data dicatat setiap detik) dengan menghitung selisih waktu antar baris pada kolom 'time' menggunakan fungsi `diff()`, lalu mengambil rata-ratanya untuk memperoleh nilai interval waktu rata-rata antar pengambilan sampel. Frekuensi sampling dihitung dengan rumus:

$$fs = \frac{1}{\Delta t} \text{ yang hasilnya ditampilkan dalam satuan Hertz (Hz).}$$

Selanjutnya, nilai frekuensi sampling tersebut digunakan untuk menentukan interval waktu antar sampel (dt). Kemudian, dilakukan integrasi numerik terhadap tiga komponen akselerasi (`acceleration_x`, `acceleration_y`, dan `acceleration_z`) untuk memperoleh kecepatan pada masing-masing sumbu (v_x , v_y , dan v_z) dengan fungsi `np.cumsum(...)`. Nilai kecepatan total atau magnitudo kecepatan (*velocity*) dihitung dari akar kuadrat jumlah kuadrat ketiga komponen kecepatan tersebut. Hasil akhirnya adalah kolom baru *velocity* yang menggambarkan besar kecepatan total pada setiap waktu.

c. Penghapusan kolom tidak relevan

```
[ ] data.drop(['acceleration_x', 'acceleration_y', 'acceleration_z'], axis = 1, inplace=True)

data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 88588 entries, 0 to 88587
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   activity    88588 non-null  int64
1   gyro_x      88588 non-null  float64
2   gyro_y      88588 non-null  float64
3   gyro_z      88588 non-null  float64
4   time        88588 non-null  float64
5   velocity    88588 non-null  float64
dtypes: float64(5), int64(1)
memory usage: 4.1 MB
```

Dilakukan *dropping* fitur yang tidak digunakan dalam proses pemodelan dataset sehingga dapat lebih berfokus pada fitur dan hasil yang nantinya akan digunakan. Kolom yang dihapus adalah `acceleration_x`, `acceleration_y`, `acceleration_z` menggunakan `data.drop` dikarenakan fitur ini sudah digantikan dengan *velocity* seperti

konversi sebelumnya sehingga tiga kolom ini bisa dihapus karena dianggap tidak relevan.

d. Labeling manual untuk velocity

```
[ ] if 'velocity' in data.columns:
    def Wvelocity(velocity):
        if velocity >= 1.27:
            return 'normal'
        else:
            return 'tidak normal'

    data['status'] = data['velocity'].apply(Wvelocity)
else:
    print("Kolom 'velocity' tidak ditemukan dalam DataFrame.")

data['status'] = data['status'].map({'normal': 1, 'tidak normal': 0})
```

Pendekatan ini menggunakan *rule-based classification*, yaitu sebuah teknik yang menetapkan label kategori berdasarkan aturan logis atau ambang nilai tertentu yang ditentukan secara eksplisit oleh pengguna. Nilai ambang (*threshold*) yang digunakan adalah 1.27 pada fitur *velocity*. Aturan yang diterapkan adalah: jika nilai *velocity* lebih besar atau sama dengan 1.27, maka data diberi label *normal*; sebaliknya, jika kurang dari nilai tersebut, diberi label *tidak normal*. Aturan ini diimplementasikan melalui fungsi *velocity*, yang kemudian diterapkan ke seluruh entri dalam kolom *velocity* menggunakan fungsi *.apply()*. Setelah klasifikasi kategori berbasis teks selesai, label tersebut dikonversi ke bentuk numerik dengan menggunakan metode *label encoding*, yaitu memetakan *normal* menjadi 1 dan *tidak normal* menjadi 0.

e. Seleksi fitur dan target, standarisasi, *encoding* target, dan data *splitting*

```
[ ] x = data[['velocity', 'gyro_x', 'gyro_y', 'gyro_z']]
    y = data[['activity', 'status']]

[ ] from sklearn.preprocessing import StandardScaler
    sc = StandardScaler()
    x_scaled = sc.fit_transform(x)
    x_scaled

↵ array([[ -1.82207202, -0.05039065, -0.00392358, -1.5419494 ],
        [ -1.82199469, -0.14349622, -0.013684  ,  0.05462392],
        [ -1.82190488, -0.72973418,  0.05764218, -1.28448139],
        ...,
        [  1.5656786  , -1.16015794, -0.26395131, -0.67066385],
        [  1.5657229  , -1.21321296,  0.34937044, -0.93910129],
        [  1.56576432,  0.09106279,  0.87409756,  0.65768097]])

[ ] from sklearn.preprocessing import LabelEncoder

    y_combined = data['activity'].astype(str) + data['status'].astype(str)
    le = LabelEncoder()
    y = le.fit_transform(y_combined)

[ ] from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(x_scaled,
                                                         y, test_size=0.20, random_state=100)
    X_train.shape, X_test.shape, y_train.shape, y_test.shape

↵ ((70870, 4), (17718, 4), (70870,), (17718,))
```

Seleksi fitur dan target dengan memilih kolom-kolom yang akan digunakan sebagai masukan (x) dan keluaran (y) pada model. Dalam hal ini, fitur yang dipilih meliputi *velocity*, *gyro_x*, *gyro_y*, dan *gyro_z*, yang merupakan data numerik dari sensor. Sedangkan untuk target, digunakan kombinasi dua kolom, yaitu *activity* dan *status*, yang berisi label aktivitas dan status kondisi data (normal atau tidak normal). Pemisahan ini bertujuan untuk membedakan atribut input dan output dalam pipeline pemodelan. Proses normalisasi atau standarisasi fitur numerik menggunakan *StandardScaler*. *StandardScaler* bekerja dengan mengubah data sehingga memiliki distribusi dengan rata-rata nol dan standar deviasi satu. Selanjutnya, dilakukan *encoding* label target, karena data kategori seperti *activity* dan *status* tidak dapat langsung digunakan dalam algoritma *machine learning*. Kedua kolom tersebut terlebih dahulu digabung menjadi satu label kategori baru dengan mengubahnya ke bentuk *string*, kemudian dilakukan *encoding* menggunakan *LabelEncoder* untuk mengonversi

setiap kombinasi label menjadi nilai numerik unik. Proses ini dikenal sebagai *label encoding* dan umum dilakukan untuk klasifikasi multi-kelas. Tahap terakhir adalah pembagian data latih dan uji (data splitting) menggunakan fungsi `train_test_split`. Data input yang telah dinormalisasi (`x_scaled`) dan label hasil encoding (`y`) dibagi menjadi dua bagian, yaitu 80% untuk data latih dan 20% untuk data uji.

3. *Exploratory Data Analysis (EDA)*

a. *Boxplot* kolom *activity* dan *velocity*

```
[ ] import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(x='activity', y='velocity', data=data)
plt.title("Distribusi velocity per activity")
plt.show()
```

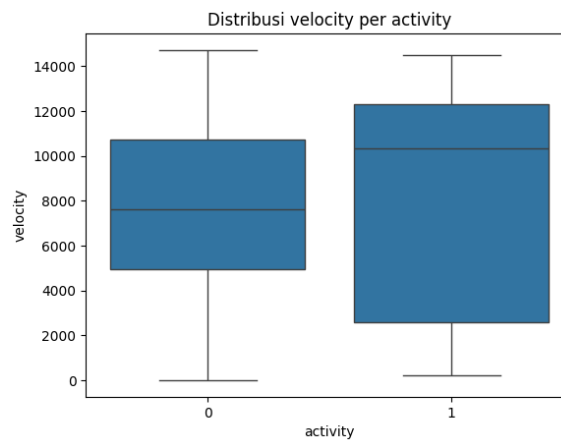
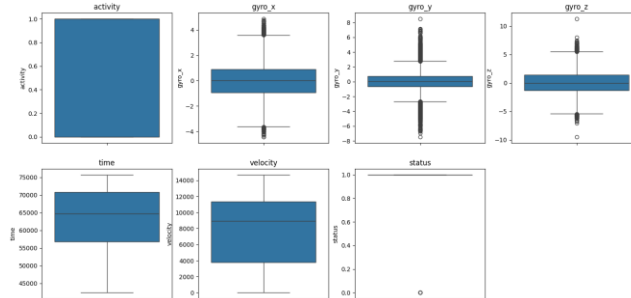


Diagram boxplot ini memberikan gambaran distribusi kecepatan pada setiap jenis aktivitas. Melalui boxplot ini, dapat dilihat apakah terdapat perbedaan distribusi *velocity* yang signifikan antar kategori aktivitas seperti berjalan, duduk, atau berlari.

b. *Boxplot* semua kolom yang digunakan

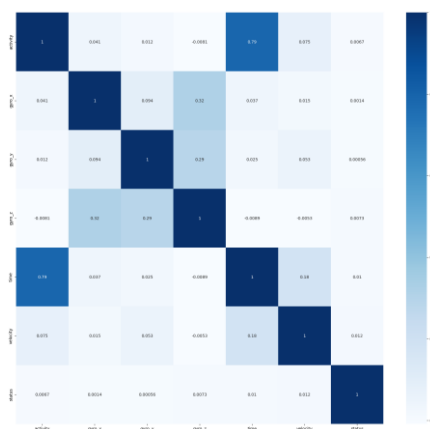
```
[ ] plt.figure(figsize=(18,18))
    i=1
    for x in data.columns:
        plt.subplot(4,4,i)
        sns.boxplot(y=data[x])
        plt.title(x)
        i+=1
```



Kolom yang masuk adalah time, status, activity velocity, gyro_x, gyro_y, dan gyro_z. Tujuan dari visualisasi ini adalah untuk mendeteksi adanya pencilan (*outlier*), sebaran data yang tidak merata, dan potensi ketidakseimbangan skala antar fitur.

c. *Heatmap* korelasi antar fitur

```
[ ] plt.figure(figsize=(20,18))
    sns.heatmap(data.corr(), cmap='Blues', annot=True)
    plt.show()
```

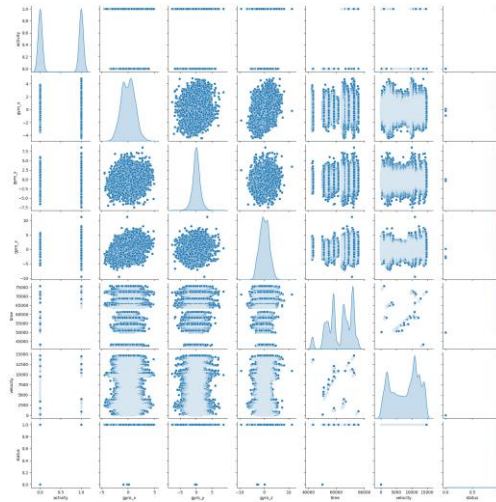


Heatmap korelasi menggunakan koefisien korelasi Pearson. Metode ini digunakan untuk mengevaluasi kekuatan dan arah hubungan linier antara fitur-fitur yang ada.

Nilai korelasi mendekati +1 atau -1 menunjukkan hubungan linier yang kuat, sementara nilai mendekati 0 menunjukkan tidak adanya hubungan linier.

a. *Pairplot*

```
[ ] sns.pairplot(data, diag_kind='kde')
```



Selanjutnya menggunakan metode visualisasi gabungan scatterplot dan histogram untuk setiap pasangan fitur. Tujuan dari metode ini adalah untuk mengamati hubungan dua dimensi antar variabel serta pola distribusi univariat.

4. Modeling

```
[ ] from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(splitter='best', criterion='entropy',
                           min_samples_split=3, min_samples_leaf=3)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)

[ ] from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
print(classification_report(y_test, y_pred_dt))
dt_ac = accuracy_score(y_test, y_pred_dt)
print("Akurasinya", dt_ac)
```

	precision	recall	f1-score	support
1	1.00	1.00	1.00	8898
2	1.00	1.00	1.00	8820
accuracy			1.00	17718
macro avg	1.00	1.00	1.00	17718
weighted avg	1.00	1.00	1.00	17718

Akurasinya 0.9971780110621966

Algoritma *Decision Tree* digunakan sebagai metode klasifikasi dengan parameter `criterion='entropy'`, `min_samples_split=3`, dan `min_samples_leaf=3`. Model dilatih menggunakan data latih `X_train` dan `y_train`, lalu diuji terhadap data `X_test`. Pemisahan simpul pohon dilakukan berdasarkan nilai informasi maksimum menggunakan kriteria *entropy*. Evaluasi performa dilakukan menggunakan metrik `accuracy_score` dan `classification_report` dari pustaka `sklearn.metrics`. Hasil prediksi dibandingkan dengan label sebenarnya (`y_test`) untuk menghitung akurasi serta nilai *precision*, *recall*, dan *f1-score*.

5. Visualisasi *Decision Tree*

```
[ ] from sklearn.tree import export_graphviz
    import graphviz

    feature_names = ['velocity', 'gyro_x', 'gyro_y', 'gyro_z']
    dot_data = export_graphviz(
        dt,
        out_file=None,
        feature_names=feature_names,
        class_names=class_labels, # ← pakai dari encoder
        filled=True,
        rounded=True,
        special_characters=True
    )
    graph = graphviz.Source(dot_data)
    graph.render("decision_tree")
    graph
```

Untuk memvisualisasikan struktur pohon keputusan yang telah dilatih, digunakan fungsi `export_graphviz` dari pustaka `sklearn.tree` dan `graphviz`. Visualisasi ini bertujuan untuk memperlihatkan bagaimana model membuat keputusan berdasarkan fitur *input*. Nama-nama fitur (`velocity`, `gyro_x`, `gyro_y`, dan `gyro_z`) dimasukkan sebagai parameter `feature_names`, sementara label kelas diatur melalui parameter `class_names`. Visualisasi dihasilkan dalam format grafis menggunakan `graphviz.Source`, yang menyajikan setiap simpul pohon beserta nilai *impurity*, jumlah sampel, serta aturan pembagian. Pengaturan `filled=True` digunakan untuk memberi warna berdasarkan dominasi kelas, sedangkan `rounded=True` memperhalus bentuk simpul untuk meningkatkan keterbacaan.

6. Aturan *Decision Tree*

```
[ ] from sklearn.tree import _tree
feature_names = ['velocity', 'gyro_x', 'gyro_y', 'gyro_z']
def tree_to_rules(tree, feature_names):
    tree_ = tree.tree_
    feature_name = [
        feature_names[i] if i != _tree.TREE_UNDEFINED else "undefined!"
        for i in tree_.feature
    ]
    print("Aturan Decision Tree:")

    def recurse(node, depth):
        indent = "  " * depth
        if tree_.feature[node] != _tree.TREE_UNDEFINED:
            name = feature_name[node]
            threshold = tree_.threshold[node]
            print(f"{indent}Jika {name} <= {threshold:.2f}:")
            recurse(tree_.children_left[node], depth + 1)
            print(f"{indent}Jika {name} > {threshold:.2f}:")
            recurse(tree_.children_right[node], depth + 1)
        else:
            print(f"{indent}Kelas: {np.argmax(tree_.value[node])}")

    recurse(0, 0)

    tree_to_rules(dt, feature_names=feature_names)
```

Aturan Decision Tree:
Jika velocity <= 0.38:
 Jika velocity <= -0.91:
 Jika velocity <= -1.41:

Proses ini dilakukan dengan memanfaatkan atribut internal dari model *Decision Tree* yang telah dilatih, melalui modul `_tree` dari `sklearn.tree`. Fungsi `tree_to_rules()` dibuat untuk menelusuri setiap simpul dalam struktur pohon secara rekursif. Pada setiap simpul, fungsi memeriksa fitur mana yang digunakan sebagai kriteria pemisahan, serta nilai ambang (*threshold*) yang menentukan arah percabangan. Jika simpul bukan simpul daun, maka aturan berupa kondisi seperti “jika fitur $\leq$ threshold” akan dicetak, dan proses dilanjutkan ke simpul kiri dan kanan. Jika simpul merupakan simpul daun, maka kelas hasil prediksi ditampilkan berdasarkan nilai dominan pada *node* tersebut.

3.4 Kodingan Arduino

Kode full : [[Kode Arduino Webserver](#)]

```
String getAccReadings() {
    mpu.getEvent(&a, &g, &temp);
    // Get current acceleration values
    accX = a.acceleration.x;
    accY = a.acceleration.y;
    accZ = a.acceleration.z;
    velocity = sqrt(abs(accX) * abs(accX) + abs(accY) * abs(accY) + abs(accZ) * abs(accZ)) - 9.8;
    readings["accX"] = String(accX);
    readings["accY"] = String(accY);
    readings["accZ"] = String(accZ);
    readings["velocity"] = String(velocity);
    String accString = JSON.stringify (readings);
    return accString;
}
```

Fungsi ini membaca data akselerasi mentah (a.acceleration.x/y/z) dari MPU. Namun, perhitungan *velocity* di sini sangat berbeda: ia menghitung *magnitude* (panjang vektor) dari akselerasi dengan $\sqrt{\text{abs}(\text{accX}) * \text{abs}(\text{accX}) + \dots}$, lalu mengurangnya dengan 9.8. Ini adalah perhitungan kecepatan terintegrasi dari percepatan, serta lebih seperti indikasi kekuatan guncangan atau percepatan total pada saat itu, yang kemudian dikurangi sebuah offset, dan hasilnya diformat menjadi JSON.

```
String getGyroReadings(){
    mpu.getEvent(&a, &g, &temp);

    float gyroX_temp = g.gyro.x;
    if(abs(gyroX_temp) > gyroXerror) {
        gyroX += gyroX_temp/50.00;
    }

    float gyroY_temp = g.gyro.y;
    if(abs(gyroY_temp) > gyroYerror) {
        gyroY += gyroY_temp/70.00;
    }

    float gyroZ_temp = g.gyro.z;
    if(abs(gyroZ_temp) > gyroZerror) {
        gyroZ += gyroZ_temp/90.00;
    }

    readings["gyroX"] = String(gyroX);
    readings["gyroY"] = String(gyroY);
    readings["gyroZ"] = String(gyroZ);

    String jsonString = JSON.stringify(readings);
    return jsonString;
}
```

Fungsi `getGyroReadings()` ini bertugas untuk mendapatkan data giroskop dari sensor MPU6050 dan memprosesnya untuk pelacakan orientasi. Fungsi ini membaca nilai giroskop mentah (g.gyro.x/y/z), menerapkan filter noise sederhana menggunakan `gyroXerror` dan sejenisnya, kemudian mengintegrasikan (menambahkan secara kumulatif) nilai-nilai tersebut ke variabel `gyroX`, `gyroY`, `gyroZ` untuk mendapatkan

perkiraan sudut rotasi. Hasil gyroX/Y/Z ini, bersama dengan nilai mentah giroskop, kemudian dikemas ke dalam format JSON untuk dikirimkan dan ditampilkan di antarmuka web.

```
String classify(float velocity_val, float gyro_x_val, float gyro_y_val, float gyro_z_val)
{
    int kelas = -1; // Default jika tidak ada kelas yang cocok
    if (velocity_val <= 0.38) {
        if (velocity_val <= -0.91) {
            if (velocity_val <= -1.41) {
                if (velocity_val <= -1.61) {
                    if (velocity_val <= -1.77) {
                        if (velocity_val <= -1.82) {
                            kelas = 0;
                        } else { // velocity_val > -1.82
                            kelas = 1;
                        }
                    } else { // velocity_val > -1.77
                        if (velocity_val <= -1.62) {
                            kelas = 2;
                        } else { // velocity_val > -1.62
                                kelas = -1;
                            }
                    }
                } else {
                    kelas = 1;
                }
            } else {
                kelas = 0;
            }
        } else {
            kelas = -1;
        }
    } else {
        kelas = -1;
    }

    // Mengembalikan string berdasarkan kelas yang ditentukan
    if (kelas == 1) {
        return "Normal & Jalan";
    } else if (kelas == 2) {
        return "Normal & Lari";
    } else if (kelas == 0) {
        return "Abnormal";
    } else {
        return "Tidak Diketahui"; // Kasus jika kelas tidak cocok
    }
}
```

Cuplikan kode ini menampilkan awal dari fungsi `classify()`, yang menjadi inti dari model Decision Tree Anda. Fungsi ini menerima nilai kecepatan (`velocity_val`, yang dalam implementasi terakhir adalah `velX` atau kecepatan sumbu X) dan nilai giroskop mentah (`gyro_x_val`, `gyro_y_val`, `gyro_z_val`). Di dalamnya, terdapat serangkaian kondisi `if-else if` yang sangat spesifik dan bersarang. Berdasarkan kombinasi nilai input sensor yang memenuhi ambang batas tertentu (termasuk nilai negatif untuk `velocity_val`), fungsi ini akan menentukan dan menetapkan kelas numerik (0, 1, atau 2) yang merepresentasikan status gerak ("Abnormal", "Normal & Jalan", atau "Normal & Lari").

```

server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    request->send(LittleFS, "/index.html", "text/html");
});

server.serveStatic("/", LittleFS, "/");

server.on("/reset", HTTP_GET, [](AsyncWebServerRequest *request){
    gyroX=0;
    gyroY=0;
    gyroZ=0;
    request->send(200, "text/plain", "OK");
});

```

Cuplikan kode ini menunjukkan bagian dari fungsi `setup()` yang mengkonfigurasi server web asynchronous. `server.on("/", HTTP_GET, ...)` mengatur agar ketika klien mengakses alamat IP ESP32, file `index.html` dari LittleFS akan dikirimkan, menampilkan antarmuka web Anda. Selain itu, `server.on("/reset", HTTP_GET, ...)` mendefinisikan sebuah endpoint khusus; ketika diakses, endpoint ini akan mereset nilai giroskop yang terintegrasi (`gyroX`, `gyroY`, `gyroZ`) menjadi nol, memungkinkan pengguna untuk mengulang pelacakan orientasi dari posisi awal.

```

if ((millis() - lastTimeAcc) > accelerometerDelay) {
    String accData = getAccReadings();
    mpu.getEvent(&a, &g, &temp); // Baca sensor
    statusGerak = classify(velocity, g.gyro.x, g.gyro.y, g.gyro.z);

    if(statusGerak != lastStatus) {
        events.send(statusGerak.c_str(), "status_gerak", millis());
        lastStatus = statusGerak;
    }

    events.send(accData.c_str(), "accelerometer_readings", millis());
    events.send(accData.c_str(), "status_velocity", millis());
    lastTimeAcc = millis();
}

```

Bagian dari fungsi `loop()` ini bertanggung jawab untuk mengatur jadwal pembacaan dan pemrosesan data akselerometer. Setiap kali `accelerometerDelay` (200 milidetik) berlalu, kode akan memanggil fungsi `getAccReadings()` untuk memperbarui data sensor dan kecepatan. Selanjutnya, ia akan memanggil `classify()` dengan nilai kecepatan dan giroskop terbaru untuk menentukan `statusGerak` perangkat. Jika

statusGerak berubah dari pembacaan sebelumnya (lastStatus), atau jika ada pembacaan akselerometer baru, data ini akan dikirimkan ke klien web yang terhubung melalui Server-Sent Events (SSE).

3.5 Prosedur

Berikut adalah prosedur uji coba dari hasil prototipe G-Track

1. Persiapan Prototipe:
 - a. Pastikan sensor MPU terpasang dengan benar dan orientasi yang konsisten di sepatu (sumbu X selalu mengarah ke depan, Y ke samping, Z ke atas). Orientasi yang tidak konsisten akan memengaruhi hasil.
 - b. Isi penuh baterai prototipe.
 - c. Nyalakan prototipe.
2. Koneksi Sistem:
 - a. Di Komputer/Laptop, buka Arduino IDE dan pastikan Serial Monitor terbuka (baud rate 115200).
 - b. Tunggu ESP32 terhubung ke jaringan Wi-Fi, kemudian akan melihat alamat IP di Serial Monitor.
 - c. Di Smartphone atau browser PC, akses alamat IP tersebut (misal: <http://192.168.1.12>). Maka akan melihat antarmuka web.
3. Kalibrasi MPU Awal:
 - a. Letakkan sepatu di permukaan yang benar-benar diam dan datar selama 5-10 detik. Jangan disentuh atau digerakkan.
 - b. Amati Serial Monitor saat proses `initMPU()` berjalan. Pada serial monitor akan tampil nilai `X=...`, `Y=...`, `Z=...`
 - c. Setelah kalibrasi selesai, klik tombol "RESET" di antarmuka web Anda. Ini akan mereset nilai `velX/Y/Z` dan `gyroX/Y/Z` yang terintegrasi ke nol, memulai pelacakan dari kondisi diam.
4. Uji Kondisi Diam (*Baseline*):
 - a. Biarkan sepatu diam total di permukaan datar selama 5 detik.
 - b. Amati di Serial Monitor (DEBUG output):

- DEBUG -- Accel: Seharusnya semua sumbu mendekati 0.000. Kecuali Z karena harus melawan gaya gravitasi.
 - DEBUG -- Gyro: Seharusnya semua sumbu mendekati 0.000.
 - DEBUG -- Hasil Klasifikasi: Seharusnya konsisten menunjukkan "Normal & Jalan".
- c. Amati di Web Interface: KECEPATAN harus menunjukkan angka mendekati 0.00. STATUS GERAK harus stabil di "Normal & Jalan".
5. Uji Gerakan "Normal & Jalan" (Walking):
- a. Mulai berjalan dengan gaya normal dan stabil.
 - b. Amati di Serial Monitor:
 - DEBUG -- Velocity X-axis: Fluktuasinya (biasanya positif, mungkin sedikit negatif saat kaki mendarat/mengayun mundur). Apakah mencapai rentang yang wajar untuk berjalan (misal, 0.5 - 1.27 m/s)?
 - DEBUG -- Gyro: Perhatikan pola puncaknya saat melangkah.
 - DEBUG -- Hasil Klasifikasi: Harapan: Mayoritas menampilkan "Normal & Jalan".
 - c. Amati di *Web Interface*: KECEPATAN harus menunjukkan angka yang berubah sesuai langkah. STATUS GERAK harus menampilkan "Normal & Jalan".
6. Uji Gerakan "Normal & Lari" (*Running*):
- a. Mulai berlari.
 - b. Amati di Serial Monitor:
 - DEBUG -- Velocity X-axis: Nilainya akan jauh lebih tinggi dan konsisten positif dibanding berjalan
 - DEBUG -- Gyro: Perhatikan pola yang berbeda dan mungkin lonjakan yang lebih tinggi dari berjalan.
 - DEBUG -- Hasil Klasifikasi: Harapan: Menampilkan "Normal & Lari".
 - c. Amati di Web Interface: KECEPATAN harus menunjukkan angka yang lebih tinggi. STATUS GERAK harus menampilkan "Normal & Lari".

7. Uji Gerakan "Abnormal":

- a. Lakukan gerakan yang tidak biasa/drastis, seperti:
 - Menggoyangkan sepatu dengan keras secara acak.
 - Memutar sepatu tiba-tiba.
 - Mencoba berjalan seperti orang dengan penyakit pada lututnya
 - Mengayunkannya dengan cepat ke arah belakang (untuk mencoba menghasilkan $velX$ yang negatif signifikan).
- b. Amati di Serial Monitor:
 - `DEBUG -- Hasil Klasifikasi: Harapan: Menampilkan "Abnormal"`.
- c. Amati di Web Interface: STATUS GERAK harus menampilkan "Abnormal".

8. Pencatatan Hasil:

- a. Untuk setiap uji (Diam, Jalan, Lari, Abnormal), catat hasilnya:
 - Nilai tipikal velocity total, $accX/Y/Z$, $gyroX/Y/Z$ dari Serial Monitor.
 - Jalur `CLASSIFY_DEBUG` yang paling sering diambil.
 - Classification Result yang muncul.

3.6 Validasi Akurasi

Model klasifikasi utama dalam sistem ini adalah algoritma *Decision Tree*, yang dipilih karena kemampuannya untuk membuat keputusan berbasis aturan yang mudah diinterpretasikan dan ringan dijalankan di perangkat embedded seperti ESP32. Dataset yang digunakan berasal dari sumber publik yang telah dikurasi dan terdiri dari aktivitas manusia, termasuk data akselerasi dan kecepatan sudut (*gyroscope*). Setiap sampel mencakup nilai dari sensor akselerometer (x, y, z), *gyroscope* (x, y, z), dan *timestamp*, yang kemudian digunakan untuk menghitung fitur *gait velocity*. Selain itu, label target pada data terdiri dari gabungan dua klasifikasi, yaitu aktivitas (jalan/lari) dan status (normal/tidak normal), yang dikodekan menjadi empat kelas gabungan.

Untuk proses validasi, dataset dibagi menjadi data pelatihan dan data pengujian menggunakan metode *train-test split* dari pustaka `sklearn.model_selection`. Proporsi

pembagian adalah 80% untuk data pelatihan dan 20% untuk data pengujian. Sebelum digunakan, data input (X) dinormalisasi menggunakan metode StandardScaler, sedangkan label target (y) dikodekan menggunakan LabelEncoder untuk memfasilitasi proses pelatihan model klasifikasi multikelas.

Evaluasi performa model dilakukan menggunakan metrik klasifikasi utama berikut:

a. Accuracy

$$Accuracy = \frac{TP + FP}{(TN + TP + FN + FP)}$$

Akurasi mengukur proporsi keseluruhan prediksi yang benar terhadap total prediksi

b. Precision

$$precision = \frac{TP}{TP + FP}$$

Presisi mengukur seberapa banyak dari hasil positif yang diprediksi benar-benar positif.

c. Recall

$$Recall = \frac{TP}{TP + FN}$$

Recall menghitung sejauh mana model mampu menangkap semua data yang relevan dalam satu kelas.

d. F1-Score

$$F1 - Score = 2 \times \frac{(Precision \times Recall)}{(Precision + Recall)}$$

F1-Score memberikan keseimbangan antara precision dan recall.

3.7 Performa

Analisis performa pada sistem G-Track mencakup evaluasi efisiensi proses klasifikasi, kestabilan output prediksi, kecepatan pemrosesan data sensor, serta respons sistem secara keseluruhan dalam kondisi real-time. Performa sistem dimulai dari proses akuisisi data sensor menggunakan MPU-6050, sebuah sensor IMU (*Inertial Measurement Unit*) yang menggabungkan akselerometer dan *gyroscope* dalam satu modul. Sensor ini mengukur percepatan linear (a) dan kecepatan sudut (ω) dalam tiga dimensi (x, y, z), yang kemudian digunakan untuk menghitung kecepatan (*velocity*) dengan pendekatan numerik integrasi. Frekuensi sampling data dihitung sebagai berikut:

$$fs = \frac{1}{\Delta t}$$

Nilai ini digunakan untuk menghitung waktu antar sampel, sehingga memungkinkan integrasi percepatan menjadi kecepatan dengan:

$$v(t) = \int a(t)dt \approx \sum a_i \cdot \Delta t$$

Pengujian dilakukan dalam tiga kondisi utama: normal-jalan, normal-lari, serta satu kondisi tambahan yaitu gerakan abnormal. Hasil pengujian menunjukkan bahwa sistem mampu:

- Menjaga kestabilan klasifikasi saat tidak ada gerakan (*velocity* mendekati nol).
- Mengklasifikasikan gerakan berjalan dengan akurasi tinggi (*velocity* antara 0.5–1.27 m/s).
- Mendeteksi lari (*velocity* >1.27 m/s) secara konsisten.
- Mengenali pola gerak abnormal seperti gerakan acak, rotasi mendadak, atau simulasi pasien osteoarthritis dengan baik.

3.8 Komunikasi Data

Sistem G-Track mendukung komunikasi data secara nirkabel berbasis webserver lokal yang dijalankan di ESP32, memungkinkan pengguna untuk memantau aktivitas dan status gerakan secara langsung melalui *browser*, tanpa memerlukan aplikasi tambahan.

Arsitektur komunikasi data dibagi menjadi tiga bagian utama:

1. Sensor ke ESP32: komunikasi menggunakan protokol I2C (*Inter-Integrated Circuit*), yang terdiri dari dua jalur utama: SCL (*clock*) dan SDA (*data*). Data percepatan dan kecepatan sudut dikirimkan dari MPU-6050 ke ESP32 secara periodik.
2. ESP32 ke Web *Interface*: ESP32 berfungsi sebagai server lokal yang dapat diakses melalui alamat IP-nya. Komunikasi ini menggunakan protokol HTTP dengan *Server-Sent Events* (SSE), yang memungkinkan data terkirim secara *real-time* dari server ke klien (*browser*).
3. Format Data: data dikirim dalam bentuk JSON untuk efisiensi *parsing* dan kompatibilitas antarmuka.

Komunikasi data dalam sistem G-Track dirancang untuk memastikan pertukaran informasi secara *real-time* antara sensor, mikrokontroler, dan pengguna. Komunikasi ini mencakup dua jalur utama, yaitu antara sensor MPU-6050 dan mikrokontroler ESP32 menggunakan protokol I2C, serta antara ESP32 dan pengguna melalui webserver berbasis Wi-Fi. Sensor MPU-6050 mengukur percepatan dan kecepatan sudut dalam tiga sumbu (x, y, z), lalu mengirimkannya ke ESP32 secara berkala (interval 200 ms). Protokol I2C menggunakan dua pin, yaitu SDA dan SCL, yang terhubung ke GPIO 21 dan GPIO 22 pada ESP32. Setelah klasifikasi dilakukan, hasilnya dikirim ke antarmuka pengguna melalui server web lokal yang dijalankan oleh ESP32. Sistem menggunakan protokol HTTP dengan metode *Server-Sent Events* (SSE) untuk mengirim data secara *real-time* ke browser pengguna. Antarmuka web terdiri atas file HTML, CSS, dan JavaScript yang disimpan dalam file system internal ESP32 (LittleFS). Klien dapat mengaksesnya melalui browser dengan alamat IP lokal ESP32. Selain itu, endpoint `/reset` disediakan untuk mengatur ulang nilai sensor.

IV. PEMBAHASAN

4.1 Perangkat Lunak Sistem G-Track

4.1.1 Model Decision Tree

Perancangan proyek G-Track sebagai sepatu pintar untuk menganalisis pola dan parameter *gait* telah berhasil dilakukan. Sistem ini mampu mengklasifikasikan subjek normal (tanpa osteoarthritis) ke dalam kelas *normal* berdasarkan nilai ambang (*threshold*) *gait velocity*, baik dalam kondisi berjalan maupun berlari, dengan menggunakan algoritma *Decision Tree*. Metode pembelajaran mesin ini dipilih karena kemampuannya dalam mengklasifikasikan data berdasarkan aturan logika yang sederhana namun tetap efektif. Selain itu, model ini juga mudah diinterpretasikan, sehingga sangat sesuai untuk diterapkan pada sistem embedded. Hal ini menjadi penting mengingat perangkat keras yang digunakan adalah ESP32, yang operasionalnya terbatas dalam lingkungan Arduino IDE dengan pemrograman berbasis C++.

Model *Decision Tree* dibangun menggunakan dataset publik dari Kaggle yang berisi 88.588 sampel data sensor. Data ini dikumpulkan melalui sensor akselerometer dan giroskop dari iPhone 5c selama interval 10 detik dengan frekuensi 5,4 sampel per detik. Dataset mencakup informasi seperti tanggal, waktu, ID pengguna, serta data sensor untuk tiap sumbu (gyroscope x, y, z dan accelerometer x, y, z). Setiap partisipan diklasifikasikan ke dalam dua kelas berdasarkan aktivitas *gait* yang ditangkap oleh sensor: subjek dengan aktivitas normal diberi label '0', sementara subjek dengan aktivitas *gait* yang tidak normal diberi label '1'. Dataset ini memiliki distribusi data yang seimbang, dengan 44.223 sampel untuk kelas normal dan 44.365 sampel untuk kelas tidak normal.

Dari keseluruhan fitur yang tersedia, dilakukan seleksi fitur untuk keperluan perancangan model berdasarkan urgensi pemanfaatan fitur. Fitur-fitur yang dipilih meliputi: label aktivitas, nilai sensor gyroscope (x, y, z), nilai sensor accelerometer (x, y, z), serta waktu yang digunakan untuk menghitung frekuensi sampling. Selain untuk mendeteksi pola *gait* saat berjalan dan berlari, objektif lain dari penelitian ini

adalah mengidentifikasi salah satu parameter penting dalam *gait analysis* pada osteoarthritis lutut, yaitu *gait velocity*.

Fitur *gait velocity* diperoleh dengan mengintegrasikan ketiga komponen percepatan dari sensor accelerometer dengan memperhitungkan frekuensi sampling dataset. Nilai ini kemudian ditambahkan ke dalam dataset sebagai fitur tambahan. Berdasarkan referensi dari Staab (2014), ambang batas *gait velocity* untuk subjek normal adalah $1,27 \pm 0,14$ m/s, sedangkan untuk subjek dengan osteoarthritis lutut berada pada kisaran $0,74 \pm 0,08$ m/s.

G-Track, sebagai prototipe sepatu pintar, saat ini masih berfokus pada pengukuran parameter *gait velocity*. Untuk menyederhanakan klasifikasi model, sistem tidak menggunakan rentang nilai “abu-abu” yang terdapat pada referensi, melainkan menetapkan batasan yang lebih tegas: subjek dianggap *normal* jika *gait velocity*-nya lebih dari 1,27 m/s, dan dianggap *tidak normal* jika nilainya sama dengan atau di bawah ambang tersebut. Maka untuk membuat label yang menandakan normal dan tidak normal dari *threshold velocity*, maka dibuat fitur status yang mengaplikasikan velocity dalam bentuk interger. Maka label tujuan dalam dataset telah sesuai untuk mendukung klasifikasi berbasis *Decision Tree* dalam sistem G-Track.

Kemudian setelah melakukan *cleaning* pada dataset, dilakukan penyetelan hiperparameter untuk mendapatkan performa model yang optimal. Konfigurasi parameter mempertimbangkan karakteristik data yang digunakan melalui pendekatan trial and error serta validasi silang untuk memperoleh hasil performa terbaik seperti yang tercermin pada tabel perbandingan performa. Beberapa *hyperparameter* yang dipilih dalam perancangan model adalah *splitter* dengan strategi “*best*” dilakukan agar algoritma memilih pembelahan terbaik pada setiap node, sehingga meningkatkan akurasi model. Criterion yang digunakan adalah entropy untuk mempertimbangkan informasi gain yang lebih rinci dalam pemisahan data dibandingkan gini. Nilai *min_samples_split* dan *min_samples_leaf* masing-masing ditetapkan menjadi 3 untuk menghindari *overfitting* dengan memastikan bahwa node tidak terbentuk hanya dari satu atau dua sampel.

Tabel 2. Parameter yang Dioptimalkan untuk Model

Model	Hiperparameter	Nilai	Deskripsi
Decision Tree	Splitter	Best	Strategi untuk memilih split di setiap node (best atau random)
	Criterion	Entropy	Fungsi untuk mengukur kualitas split
	Min_samples_split	3	Jumlah minimum sampel yang dibutuhkan untuk membagi internal node
	Min_samples_leaf	3	Jumlah minimum sampel yang dibutuhkan pada setiap daun (leaf)

Kemudian, untuk mengevaluasi kinerja model prediksi dan menghindari overfitting, dataset dibagi menjadi set pelatihan (80% sampel data) dan set pengujian (20% sampel data). Penelitian membandingkan metrik evaluasi umum (akurasi, presisi, *recall*, dan *F1-score*) dari machine learning yang dieksperimenkan yaitu *Support Vector Machine*, *Decision Tree*. Persamaan matematika dari metrik evaluasi kami didefinisikan di bawah ini:

$$precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 - Score = 2 \times \frac{(Precision \times Recall)}{(Precision + Recall)}$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Evaluasi performa model dilakukan dengan menggunakan *classification report*, yang mencakup tiga metrik utama: *precision*, *recall*, dan *F1-score*. Presisi didefinisikan sebagai kemampuan dari sebuah pengklasifikasi untuk mengidentifikasi titik data yang relevan. TP (*true positive*) adalah ketika sebuah observasi positif juga diprediksi sebagai positif. FP (*false positive*) adalah observasi negatif yang diindikasikan sebagai positif. *Precision* mengukur ketepatan model dalam mengklasifikasikan kelas positif.

Recall adalah pengukuran yang mendefinisikan seberapa tepat sebuah model telah mengidentifikasi kelas-kelas positif dalam dataset kita. FN (*false negative*) merepresentasikan observasi positif yang diprediksi sebagai negatif. *Recall* menilai kemampuan model dalam mendeteksi seluruh kasus yang seharusnya termasuk kelas positif.

F1-Score adalah pengukuran terhadap akurasi model pada dataset. Rata-rata tertimbang antara *precision* dan *recall*. *F1-score* digunakan sebagai indikator gabungan untuk menilai keseimbangan antara *precision* dan *recall*. Digunakan juga akurasi yaitu pengukuran yang menentukan efisiensi sebuah model dalam mengidentifikasi pola dan hubungan antar fitur dalam dataset, yaitu rasio antara nilai yang diprediksi dengan benar terhadap total jumlah evaluasi, di mana TN (*true negative*) adalah observasi negatif yang diprediksi dengan benar sebagai negatif.

Kemudian dilakukan penambahan *confusion matrix* lebih informatif dibandingkan hanya menggunakan akurasi, terutama ketika distribusi data tidak seimbang atau ketika dampak kesalahan klasifikasi berbeda untuk tiap kelas. Selain itu, *confusion matrix* divisualisasikan untuk memberikan gambaran menyeluruh terhadap distribusi prediksi benar dan salah pada setiap kelas.

Dilakukan penganalisisan hasil evaluasi model setelah dilakukan seleksi fitur dan pelatihan yang menggunakan metode *machine learning* (ML) terhadap performa model berdasarkan metrik akurasi, *F1-score*, presisi, dan *recall* menjadi penting dalam menentukan algoritma yang paling tepat untuk digunakan dalam merancang model. Pada proyek ini, dilakukan klasifikasi biner untuk memprediksi kondisi subjek, yaitu “Normal” atau “Tidak Normal” serta “Jalan” dan “Lari”. Dari

empat kelas tersebut, dilakukan penggabungan klasifikasi/pelabelan dalam rangka penyederhanaan sistem dengan Kelas 1 menandakan bahwa subjek pada keadaan Tidak Normal dan Jalan, kelas 2 yaitu Normal dan Jalan, serta kelas 3 yaitu Normal dan Lari. Tabel 2. menunjukkan perbandingan performa model machine learning yang digunakan, yaitu Decision Tree (DT).

Tabel 3. Performa Metode *Machine Learning*

Model	<i>Accuracy</i>	<i>F1-Score</i>	<i>Presicion</i>	<i>Recall</i>
<i>Decision Tree</i>	0.997	1.00	1.00	1.00

Berdasarkan hasil evaluasi performa yang ditampilkan dalam tabel, algoritma *Decision Tree* menunjukkan kinerja yang sangat baik dalam mengklasifikasikan aktivitas berjalan dan berlari berdasarkan data sensor. Model ini mencatatkan akurasi sebesar 0,997, dengan nilai *F1-Score* dan presisi masing-masing sebesar 1,00. Nilai tersebut mencerminkan keseimbangan model dalam meminimalkan kesalahan baik positif palsu (*false positive*) maupun negatif palsu (*false negative*). Selain itu, nilai *recall* yang juga mencapai 1,00 menunjukkan bahwa *Decision Tree* mampu mengenali seluruh data uji dengan sangat baik tanpa ada yang terlewat.

Secara keseluruhan, model *Decision Tree* terbukti unggul secara statistik karena mampu mendeteksi pola dengan presisi dan sensitivitas tinggi, sambil tetap mempertahankan karakteristik utamanya: cepat, ringan, serta mudah diinterpretasikan. Hal ini menjadi nilai tambah yang penting, mengingat model akan diimplementasikan pada perangkat mobile berbasis mikrokontroler, yang menuntut efisiensi sekaligus transparansi dalam logika keputusan.

Keunggulan model ini juga telah diuji lebih lanjut dengan memberikan input nilai acak pada fitur *gyroscope* (x, y, z) dan *gait velocity*. Hasilnya, model mampu mengklasifikasikan seluruh kelas dengan benar, memperkuat validitas performa statistik yang sebelumnya ditunjukkan. Berdasarkan hasil tersebut, struktur aturan dari model *Decision Tree* kemudian dicetak dan diterjemahkan ke dalam kode yang disesuaikan untuk diimplementasikan langsung pada mikrokontroler ESP32 menggunakan Arduino IDE.

4.1.2 Kode Arduino IDE

Pertama-tama, fondasi dari sistem ini adalah pengambilan data sensor dari modul MPU6050, sebuah sensor *inertial measurement unit* (IMU) yang mengukur akselerasi linier dan kecepatan sudut (giroskop). Proses ini dimulai dengan fungsi `initMPU()` saat ESP32 pertama kali dinyalakan. Fungsi ini memastikan bahwa mikrokontroler dapat berkomunikasi dengan MPU6050, yang merupakan langkah awal krusial agar sensor dapat mulai mengirimkan data gerak.

Begitu sensor aktif, fungsi `loop()` yang berjalan tanpa henti akan memicu dua operasi penting secara periodik. Salah satunya adalah pemanggilan `getGyroReadings()`, yang bertanggung jawab untuk membaca data giroskop (rotasi pada sumbu X, Y, dan Z). Fungsi ini mengambil data akselerasi (perubahan kecepatan) dari MPU6050. Data akselerasi pada sumbu X, Y, dan Z ini kemudian diproses untuk menghitung magnitudo akselerasi total. Dari magnitudo ini, nilai gravitasi (sekitar 9.8 m/s^2) dikurangkan. Tujuan dari pengurangan gravitasi ini adalah untuk mengisolasi akselerasi yang disebabkan oleh gerakan tubuh itu sendiri, bukan karena tarikan bumi. Hasil dari perhitungan ini adalah variabel *velocity*, yang dalam konteks sistem ini diinterpretasikan sebagai kecepatan gaya berjalan (*gait velocity*). Kecepatan gaya berjalan adalah metrik yang sangat penting karena kecepatan berjalan yang melambat secara signifikan sering kali merupakan indikasi adanya masalah muskuloskeletal, termasuk *osteoarthritis* (OA).

Setelah mendapatkan nilai *velocity* dan data giroskop terkini, kedua parameter ini kemudian menjadi input untuk fungsi `classify()`. Ini adalah model *Machine learning* berbasis *decision tree* telah dilatih sebelumnya. Di dalamnya terdapat serangkaian kondisi *if-else* yang kompleks, yang bekerja seperti “pohon keputusan” untuk menganalisis pola data. Berdasarkan nilai *velocity*, `gyroX`, `gyroY`, dan `gyroZ`, fungsi ini akan memutuskan kelas gerakan mana yang paling sesuai:

- a. Normal & Jalan: Mengindikasikan pola berjalan yang sehat dengan kecepatan yang memadai.
- b. Normal & Lari: Mengindikasikan pola lari yang sehat.

- c. Abnormal: Ini adalah klasifikasi yang paling relevan dengan deteksi OA. Jika gait *velocity* jatuh di bawah ambang batas tertentu (misalnya, di bawah 0.8 m/s), atau jika ada kombinasi parameter giroskop dan kecepatan yang tidak lazim, sistem akan mengklasifikasikannya sebagai “Abnormal”. Ini berfungsi sebagai bendera merah, memberikan indikasi awal bahwa mungkin ada masalah pada pola gerak pengguna, yang bisa menjadi tanda *osteoarthritis*.

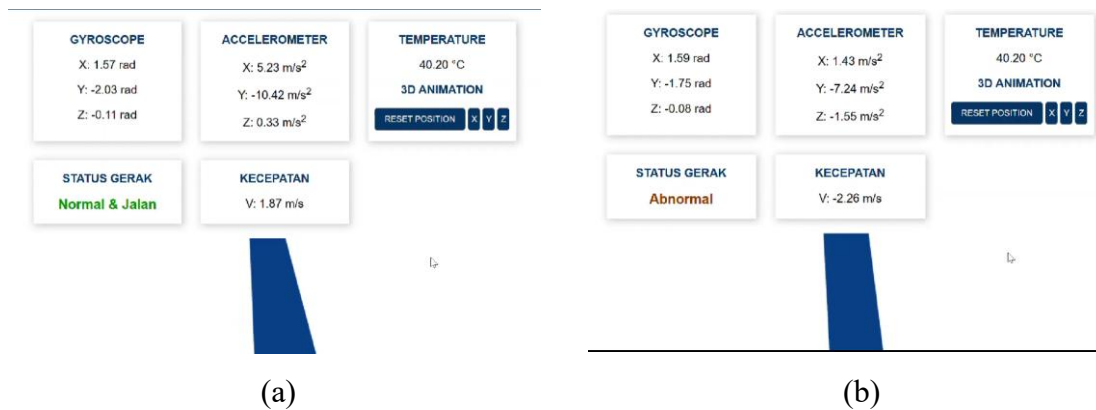
Hasil klasifikasi dari `classify()` disimpan dalam variabel `statusGerak`. Setiap kali `statusGerak` berubah dari status sebelumnya (`lastStatus`), sistem akan mengirimkan pembaruan status ini melalui *event source* ke *web server* yang dijalankan oleh ESP32. Ini memungkinkan pengguna untuk melihat status gerak mereka secara real-time di antarmuka web, menjadikannya alat yang portable dan mudah diakses. Selain itu, data mentah sensor dan status klasifikasi juga dicetak ke Serial Monitor untuk tujuan *debugging* dan pemantauan detail oleh pengembang atau analis.

4.1.3 Kode Web Server

Empat file yang berbeda digunakan untuk membangun server web yaitu sketsa Arduino, file HTML, file CSS, dan file JavaScript. File HTML, CSS dan JavaScript harus disimpan di dalam folder yang disebut `data` di dalam folder sketsa Arduino. Dalam proyek web server ESP32 dengan sensor MPU-6050 ini, sistem terdiri dari tiga komponen utama yaitu `index.html`, `style.css`, dan `script.js` yang saling terintegrasi untuk menampilkan data sensor secara *real-time* melalui browser. File `index.html` berfungsi sebagai kerangka utama halaman web, yang menyusun tampilan antarmuka pengguna berupa panel-panel untuk menampilkan data *gyroscope*, *accelerometer*, dan suhu, serta menyertakan area untuk animasi 3D. HTML ini juga menyediakan tombol *reset* yang dapat mengirim perintah ke ESP32. Kemudian, *file* `style.css` bertugas mengatur tampilan visual dari halaman agar terlihat rapi, modern, dan responsif, termasuk pewarnaan, ukuran huruf, dan layout grid agar tampilan tetap menarik di berbagai perangkat. Sementara itu,

script.js memuat logika interaktif utama yang menggunakan pustaka Three.js untuk menampilkan objek kubus 3D yang dapat berputar sesuai data *gyroscope*, serta membuka koneksi EventSource untuk menerima data dari ESP32 secara terus-menerus. Ketika data *gyroscope*, *accelerometer*, atau suhu diterima, nilai-nilai tersebut diperbarui langsung di halaman web dan animasi 3D ikut berubah mengikuti rotasi yang terdeteksi. Selain itu, ditambahkan juga hasil dari penerapan model yang dimasukkan pada sketsa arduino yang mengubah seluruh nilai accelerometer secara real time menjadi kecepatan (*gait velocity*) serta klasifikasi yang dihasilkan dari model DT yang telah dibuat menjadi status gerak. Dapat terlihat pada gambar dibawah bahwa sistem berhasil mengintegrasikan lingkup web dan sketsa arduino IDE yang sudah dilengkapi aturan DT dari model untuk menghasilkan klasifikasi gait secara real time yang memungkinkan pengguna memantau orientasi dan gerakan dari sensor MPU-6050 secara langsung melalui jaringan lokal, hanya dengan membuka halaman web dari ESP32.

Penggunaan web ini juga penentu keberhasilan integrasi sistem pada saat uji coba seluruh sistem, dan sistem menunjukkan stabilitas dalam berbagai kondisi permukaan dan kecepatan berjalan pada saat uji coba. Meski masih terdapat beberapa keterbatasan seperti sensitivitas terhadap *noise*, secara umum sistem menunjukkan potensi besar dalam perbekembangan lebih lanjut sebagai alat bantu klinis untuk skrining dini OA dan pemantauan aktivitas gait secara berkala. Dengan pengembangan lanjutan dan peningkatan fitur, sepatu pintar ini dapat menjadi solusi inovatif dalam bidang *digital health*, khususnya dalam mendukung populasi lansia untuk tetap aktif dan memantau kesehatan sendi kaum lansia secara mandiri.



Gambar 1. (a) Tampilan Server web saat status gerakan normal & jalan (b) Tampilan server web saat status gerakan abnormal

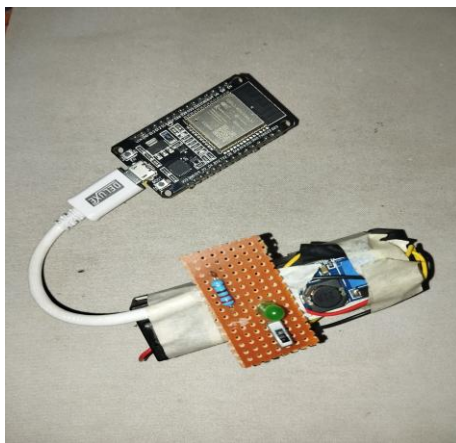
4.2 Perangkat Keras Sistem G-Track

Pembuatan sepatu pintar G-Track menggunakan beberapa komponen yakni, sensor Girooskop GY-521 MPU-6050, ESP32, baterai Li Ion 1865, kotak baterai 18650, modul charger TP4056, modul step up DC MT3608, resistor, LED, dan saklar, board PCB, kabel jumper, kabel Micro USB.

Sensor inersia digunakan untuk mendeteksi accelerometer (percepatan) dan *gyroscope* (rotasi) pada sumbu X, Y, Z saat bergerak seperti jalan ataupun lari. Kemajuan teknologi sensor inersia dalam beberapa tahun terakhir telah membuka peluang besar untuk melakukan analisis *gait* secara mobile. Penggunaan sensor ini memungkinkan pemantauan pola berjalan secara jarak jauh di lingkungan alami pengguna, sehingga meningkatkan validitas ekologis dalam penilaian fungsi fisik. Dibandingkan dengan sistem optoelektronik yang kompleks, sensor inersia menawarkan berbagai keunggulan seperti biaya rendah, fleksibilitas lokasi, waktu persiapan yang singkat, dan kemudahan penggunaan. Data dari girooskop dan akselerometer dapat dianalisis untuk mendeteksi momen penting dalam siklus berjalan secara akurat. Dari sinyal tersebut, berbagai parameter *gait* spasiotemporal dan kinematik dapat dihitung berdasarkan konfigurasi sensor dan algoritma pemrosesan yang digunakan. Berbagai penelitian telah menunjukkan bahwa algoritma yang mengolah data sensor inersia memiliki validitas dan reliabilitas yang baik hingga

sangat baik pada individu sehat, terutama untuk parameter spasiotemporal dan kinematik, meskipun terdapat keterbatasan pada metrik variabilitas dan asimetri. Oleh karena itu, sistem berbasis sensor inersia dapat menjadi alat yang menjanjikan untuk memperoleh pengukuran objektif terhadap kualitas *gait*, termasuk pada pasien dengan osteoarthritis, yang umumnya sulit dilakukan dalam pengaturan klinis konvensional (Boekesteijn et. al., 2022).

Mikrokontroler ESP32 digunakan untuk memproses data yang diperoleh dari sensor giroskop, mikrokontroler ini juga digunakan untuk menghubungkan data aktivitas ke web server untuk menampilkan data *accelerometer*, giroskop, status gerak, kecepatan, dan keadaan sensor. Baterai Li-ion 18650 digunakan sebagai sumber daya yang dapat diisi ulang. Baterai tersebut diletakkan pada kotak baterai agar aman dan praktis. Untuk mengisi ulang baterai menggunakan modul charger TP4056 yang dilengkapi proteksi terhadap *overcharge* dan *overdischarge*. Karena tegangan baterai biasanya sekitar 3.7V, maka diperlukan modul *step-up* DC MT3608 untuk menaikkan tegangan menjadi 5V agar stabil dan sesuai dengan kebutuhan ESP32. Komponen pendukung lainnya seperti resistor, LED, dan saklar digunakan untuk mengatur arus listrik, memberikan indikator visual status sistem, serta sebagai tombol *on/off* manual yang dirakit pada *PCB (Printed Circuit Board)*. Agar bisa menghidupkan ESP32 yang menggunakan *port micro USB*, maka perlu kabel *micro USB* yang dihubungkan pada modul *charger*.



Gambar 2. Modul *Prototype* G-Track

Pemasangan modul pada sepatu diletakkan pada kotak wadah agar modul lebih aman dan lebih rapih. Kemudian kotak dipasangkan dengan *strap* velcro agar mudah dilepas dan dipasang sesuai kebutuhan. Penempatan sensor dan modul secara strategis di sepatu juga diperhitungkan agar tidak mengganggu kenyamanan pengguna dan tetap menjaga validitas data gerakan. Selain itu, penggunaan kabel *micro* USB diperlukan untuk menghubungkan ESP32 ke modul *charger*, baik untuk pengisian daya maupun untuk pemrograman awal. Ke depannya, sistem ini dapat ditingkatkan dengan menambahkan fitur-fitur seperti indikator baterai, penyimpanan data menggunakan microSD, atau bahkan integrasi Bluetooth untuk aplikasi mobile. Dengan rancangan hardware yang matang seperti ini, sepatu pintar G-Track tidak hanya mampu menjalankan fungsi monitoring gerakan, namun juga menjadi platform yang andal dan fleksibel untuk riset maupun aplikasi klinis terkait analisis gait, termasuk pada pasien dengan gangguan muskuloskeletal seperti osteoarthritis.



(a)



(b)

Gambar 3. (a) Kotak Wadah (b) Kotak Wadah pada Sepatu

V. KESIMPULAN

Proyek ini adalah pengembangan sepatu pintar G-Track sebagai sarana pemantauan analisis gaya berjalan untuk deteksi dini perubahan pola *gait*, dukungan pemantauan rehabilitasi, serta peningkatan performa gerak. Sistem ini mengintegrasikan sensor MPU-6050, yang mendeteksi percepatan dan rotasi pada sumbu X, Y, Z secara akurat, dengan mikrokontroler ESP32 sebagai unit pemroses utama. Algoritma *Decision Tree* dirancang dan diterapkan untuk menganalisis data sensor dan mengidentifikasi perubahan pola *gait* yang menjadi indikator awal Osteoarthritis. Model *Decision Tree* menunjukkan kinerja yang kuat dengan akurasi 0,997, serta nilai *F1-Score*, presisi, dan *recall* masing-masing 1,00, menggunakan ambang batas *gait velocity* yang relevan untuk klasifikasi kondisi.

Fitur pemantauan *real-time* juga diimplementasikan melalui *web server* yang berjalan pada ESP32, menampilkan data giroskop, akselerometer, suhu, kecepatan *gait*, dan status gerak secara langsung pada antarmuka web. Dengan kemampuan mengklasifikasikan pola gerak seperti "Normal & Jalan" dan "Normal & Lari", serta menyediakan data objektif, sistem ini menyediakan umpan balik yang berpotensi mendukung optimalisasi performa gerak dan pencegahan cedera. Sepatu pintar G-Track menunjukkan potensi signifikan sebagai alat non-invasif yang dapat digunakan untuk skrining dini Osteoarthritis dan pemantauan aktivitas *gait* secara berkala.

VI. REFERENSI

- Al Fath, M. I., Asshidiqi, M. F., & Praja, M. B. (2018). Prototipe Sistem Monitoring Balanced Angular Momentum Seismometer Berbasis IoT. In *PROSIDING SEMINAR NASIONAL INSTRUMENTASI, KONTROL DAN OTOMASI* (pp. 341-346).
- Anshari, A., Jajat, J., Sultoni, K., Suherman, A., Ruhayati, Y., & Nuryanti, W. D. (2025). ESTIMASI KALORI EKSPENDITURE BERDASARKAN ACCELEROMETER ACTIGRAPH DAN TREADMILL. *Jambura Journal of Sports Coaching*, 7(1), 110-117.
- Amalia, P. B. A., Astuti, D., & Widyastuti, R. (2023). Analisis Faktor Risiko Terjadinya Osteoarthritis. *CoMPHI Journal: Community Medicine and Public Health of Indonesia Journal*, 4(2).
- Aviles-Cruz, C., Rodriguez-Martinez, E., Villegas-Cortez, J., & Ferreyra-Ramirez, A. (2019). Granger-causality: An efficient single user movement recognition using a smartphone accelerometer sensor. *Pattern Recognition Letters*, 125, 576-583.
- Chen, W., Chang, W., Chung, Y., & Li, Y. (2016). A wearable gait analysis system for elderly fall prediction using an MPU-6050 sensor. *Journal of Medical Systems*, 40(5), 1-8.
- Donnenfield, J. I., Karamchedu, N. P., Proffen, B. L., Molino, J., Murray, M. M., & Fleming, B. C. (2022). Predicting severity of cartilage damage in a post-traumatic porcine model: Synovial fluid and gait in a support vector machine. *Plos one*, 17(6), e0268198.
- Fraiwan, L., & Hassanin, O. (2021). Computer-aided identification of degenerative neuromuscular diseases based on gait dynamics and ensemble decision tree classifiers. *Plos one*, 16(6), e0252380.
- GBD 2021 Osteoarthritis Collaborators. (2023). Global, regional, and national burden of osteoarthritis, 1990–2020, and projections to 2050: a systematic analysis for the Global Burden of Disease Study 2021. *The Lancet Rheumatology*, 5(9), e487-e504.

- Glyn-Jones, S., Palmer, A. J. R., McLardy-Smith, R., Murray, D. W., & Barker, J. K. (2015). Osteoarthritis. *The Bone & Joint Journal*, 97-B(5), 589-597.
- Halodoc. (2020). Osteoarthritis. Diakses dari <https://www.halodoc.com/kesehatan/osteoarthritis>
- Harni, N. S. Y., Kep, M., Kom, S. K., Wenny, N. B. P., & Kep, M. (2023). *Asuhan Keperawatan Lansia dengan Osteoarthritis*. CV. Mitra Edukasi Negeri.
- Homes, R., Clark, D., Moridzadeh, S., Tosovic, D., Van den Hoorn, W., Tucker, K., & Midwinter, M. (2023). Comparison of a wearable accelerometer/gyroscopic, portable gait analysis system (LEGSYS+ TM) to the laboratory standard of static motion capture camera analysis. *Sensors*, 23(1), 537.
- Hunter, D. J., & Bierma-Zeinstra, S. (2019). Osteoarthritis. *The Lancet*, 393(10173), 1746-1759.
- Hussain, M., Al-Turjman, F., & Al-Rahmi, W. M. (2017). Smart healthcare system for real-time monitoring and analysis of human gait. *Journal of Medical Systems*, 41(9), 1-13.
- Kraus, V. B., Vail, T. P., Guilak, F., & Spector, M. (2015). Future directions in the development of therapeutic agents for osteoarthritis. *Arthritis & Rheumatology*, 67(1), 1-13.
- Li H, Hu S, Zhao R, Zhang Y, Huang L, Shi J, Li P, Wei X. Gait Analysis of Bilateral Knee Osteoarthritis and Its Correlation with Western Ontario and McMaster University Osteoarthritis Index Assessment. *Medicina (Kaunas)*. 2022 Oct 9;58(10):1419. doi: 10.3390/medicina58101419. PMID: 36295577; PMCID: PMC9610794.
- Martel-Pelletier, J., Barr, A. J., Cicuttini, F. M., Conaghan, P. G., Cooper, C., Goldring, M. B., & Pelletier, J. P. (2016). Osteoarthritis. *Nature Reviews Disease Primers*, 2(1), 1-19.
- Neogi, T., Zhang, Y. (2013). Epidemiology of Osteoarthritis. *Rheumatic Disease Clinics of North America*, 39(1), 1–19.
- Ningsih, R. (2021). *Faktor-faktor yang Berhubungan dengan Kejadian Osteoarthritis Lutut*. Repository UM Surabaya.

- Nisa, N. K., & Prabandari, Y. S. (2018). Prevalensi Osteoarthritis di Indonesia. *Jurnal Ilmu Kesehatan Masyarakat*, 9(3), 198-204.
- Olyanasab, A., & Annabestani, M. (2024). Leveraging machine learning for personalized wearable biomedical devices: A review. *Journal of personalized medicine*, 14(2), 203.
- Rodiah, S., Risna, A., Azahra, A., & Agustin, M. N. (2024). Pola Berjalan Manusia. *Journal Human Resource Strengthening*, 1(01), 54-58.
- Sari, A. A. (2020). Prevalensi Osteoarthritis pada Lanjut Usia di Puskesmas Wilayah Kota Banjarmasin. Repository ULM.
- Staab, W., Hottowitz, R., Sohns, C., Sohns, J. M., Gilbert, F., Menke, J., Niklas, A., & Lotz, J. (2014). Accelerometer and gyroscope based gait analysis using spectral analysis of patients with osteoarthritis of the knee. *Journal of physical therapy science*, 26(7), 997–1002. <https://doi.org/10.1589/jpts.26.997>
- Sugandh, F. N. U., Chandio, M., Raveena, F. N. U., Kumar, L., Karishma, F. N. U., Khuwaja, S., ... & Sugandh, F. (2023). Advances in the management of diabetes mellitus: a focus on personalized medicine. *Cureus*, 15(8).
- Sukmana, D. J., & Suryajaya, T. (2019). Patofisiologi Osteoarthritis. *Jurnal Kesehatan Andalas*, 8(2), 23-28.
- Tanpure, S., Phadnis, A., Nagda, T., Rathod, C., Kothurkar, R., & Chavan, A. (2023). Gait variability and biomechanical distinctions in knee osteoarthritis: Insights from a 3D analysis in an adult elderly cohort. *Journal of orthopaedics*, 49, 172–179. <https://doi.org/10.1016/j.jor.2023.12.011>
- Trabassi, D., Serrao, M., Varrecchia, T., Ranavolo, A., Coppola, G., De Icco, R., ... & Castiglia, S. F. (2022). Machine learning approach to support the detection of Parkinson's disease in IMU-based gait analysis. *Sensors*, 22(10), 3700.
- Wang, X., & Chen, J. (2018). Wearable sensor-based human gait analysis for elderly health monitoring: A review. *IEEE Sensors Journal*, 18(22), 9181-9199.